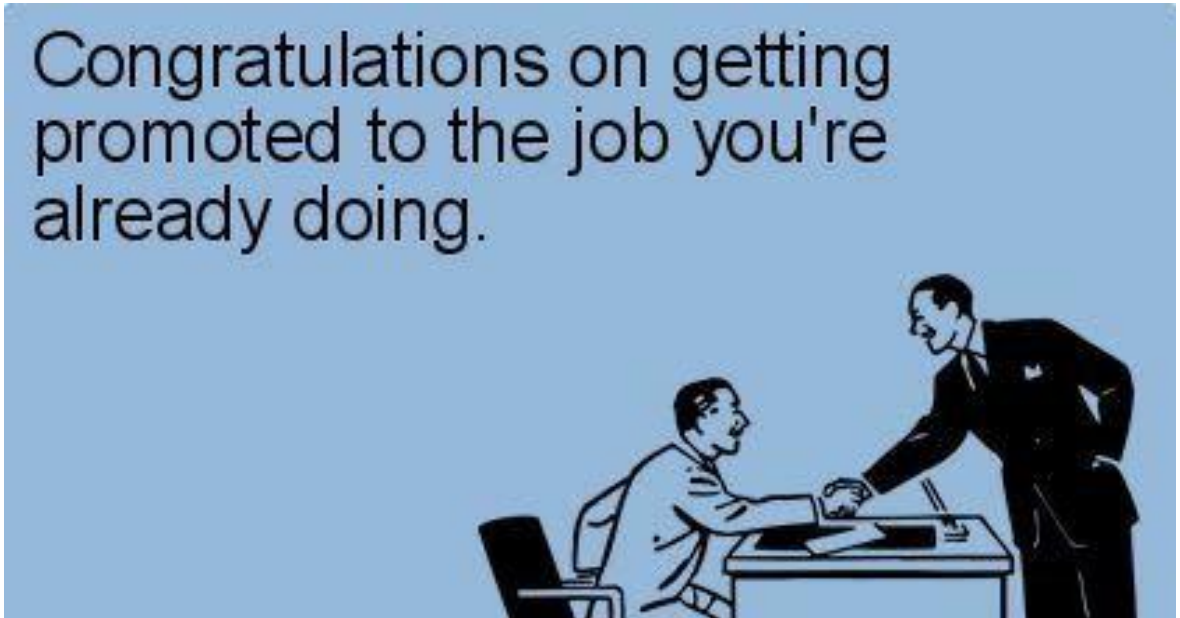
Promotion as a SWE is critical

... but it’s *painfully* misunderstood 🤷‍♂️



I’m Alex

- My goal: **Make you a better engineer who gets rewarded by level** 🚀
- Previously a Tech Lead at Course Hero, Meta, and Robinhood
- Prior to Taro, making $750k/year as a top TL leading 15+ engineers
- Coached dozens of engineers to ⚡ promotions at top companies

Junior 🤓

Meta

E3

$192,702

Alex's Mentorship 🪴

1 year promotion

10+ Engineers

Mid-Level 🙂

Meta

E4

$306,684

*A few mentees got promoted in just 6 months!

Mid-Level 🙂

Meta

E4

$306,684

Alex's Mentorship 🪴

1 year promotion

5+ Engineers

Senior 😎

Meta

E5

$475,444

*Almost all other E4 mentees got promoted in 1.5 years!

Junior 🤓

Meta

E3

$192,702

Alex's Mentorship 🪴

3 years

5+ Engineers

Senior 😎

Meta

E5

$475,444

Greatly Exceeds Expectations 🚀

*I have raised multiple new-grads to tech leads!

Promotion Is Underrated

- Everyone just tries to switch jobs when things go wrong
  - The problem is that interviewing is parasitic
- If you work at a top company, promotion is generally a good option
  - Your level is a floor
- Promotion is conducive to growth unlike interview prep

Interviewing

- Work with irrelevant made-up problems
- Memorize facts to fake a shallow understanding
- Pretend to be interested in people and their company
- Ship throwaway software with minimal quality

Promotion

- Work on actual projects demonstrating business value
- Develop deep understanding of technical and business needs
- Build genuine relationships with peers
- Ship high-quality software to show longevity

Work on actual projects demonstrating business value		**Prioritization, data analysis, experimentation**
Build genuine relationships with peers		**Empathy, networking, communication**
Develop deep understanding of technical and business needs		**Product sense, user research, infrastructure**
Ship high-quality software to show longevity		**System design, debugging, quality control**

Objectives

- 💡 Understand how promotion **actually works**, particularly at top companies
- 🧠 Have a framework to understand the **senior engineering career ladder**
- 📃 Define a **clear plan** to get to the next level
- ✅ Get **maximum credit** for every project
- 🚀 Master your performance review to produce the **best promo packet**

You Can’t Job Hop To Success

...At Least Not Forever

- It is risky for companies to take in an engineer at a level they have never had
  - This is more prominent in a bad market
- Most engineers will hit a wall at Senior/Staff
- Interview questions tend to converge to actual experience at senior+

Asking 7 Levels Of Engineers The Most Important Skill

Save

I asked every step of the IC engineering ladder a simple question: "What is the most important skill for success at your level?" The result is an engineer from each of the 7 levels sharing their candid reflections about the most important skill, along with advice. Companies represented: Meta, Slack, Qualcomm, Gusto, Instacart, Cruise, and Pinterest.

Entry-Level Engineer (L3) Describes Most Important Skill - Uriel Sejas

808 Views 17 Likes

Mid-Level Engineer (L4) Describes Most Important Skill - Dipika Tiwari

510 Views 14 Likes

Senior Engineer (L5) Describes Most Important Skill - Richard Chen

949 Views 24 Likes

Staff Engineer (L6) Describes Most Important Skill - Sammy Nguyen

602 Views 7 Likes

Senior Staff Engineer (L7) Describes Most Important Skill - Kaushik Gopal

1.2K Views 20 Likes

Senior Staff Engineer at Cruise Shares The Most Important Skill -...

821 Views 15 Likes

Sam Nguyen · 2nd

Engineering at Gusto

Kaushik Gopal ·

Senior Staff | Head of Caper

Andrew Zhai · 1st

Experience

Software Engineering
Gusto
Jan 2017 - Present · 7 yrs 3 mos

~~Instacart~~
8 yrs 8 mos

Principal Engineer
Mar 2024 - Present · 1 mo

Head of Caper Software (
Mar 2022 - Present · 2 yrs 1 m

Senior Staff Engineer
Apr 2019 - Present · 5 yrs

Distinguished Engineer, Advan
Pinterest · Full-time
Feb 2014 - Mar 2023 · 9 yrs 2 mos

How To Become A 10x Engineer 💪

Step 1: Find an organization that pushes you to get better and rewards you for it 

Step 2: Be hungry, consistently add value over a long period of time 🙌

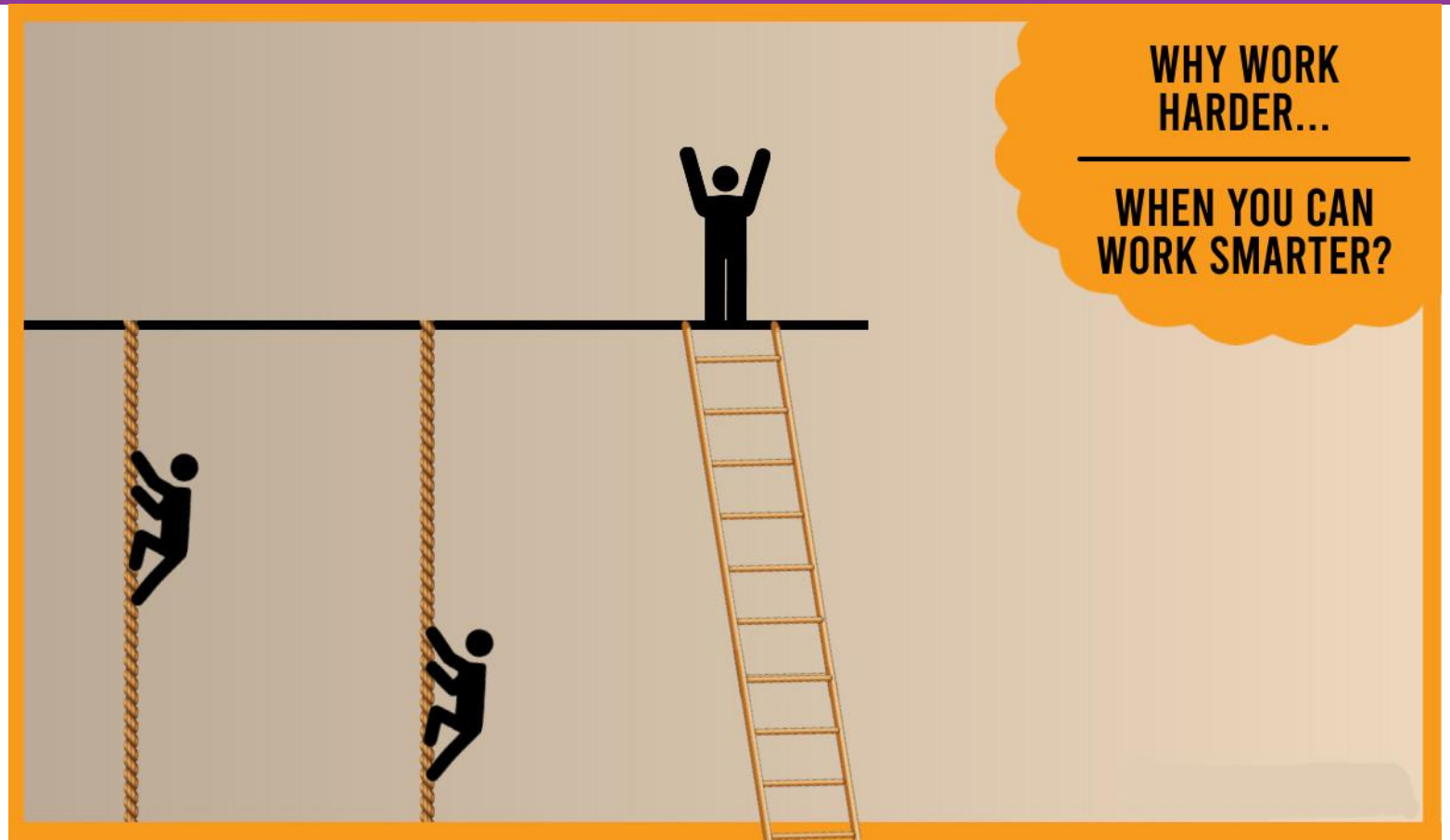
Step 3: Grow like a 🚀

So Why Does It Take So Long?

- Understand company culture 💡
- Cultivate strong relationships 🤝
- Deep technical know-how 👨‍💻
- Holistic view of problems ❌
- Product intuition 🤔

In order to grow, you must plant roots.

The Key To Understanding Promotion



Promotion Is About *Behavior*, Not Output 🔑

- What separates junior engineers from very senior engineers is *how* the senior+ SWEs use their time
- Every single day is full of *thousands* of decisions
  - Staff+ engineers overwhelming get these decisions right
- Think ***multiplicatively***

Junior 🤓

Meta

E3

$192,702

8 hour days
5 commits
per week

17x TC Increase

Distinguished 🤯

Meta

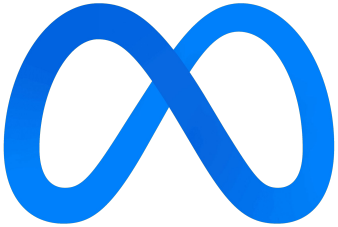
E9

$3,251,000

136 [illegible]?
85 com[illegible]
per w[illegible]k?

Software = *Disproportionate* Outcomes

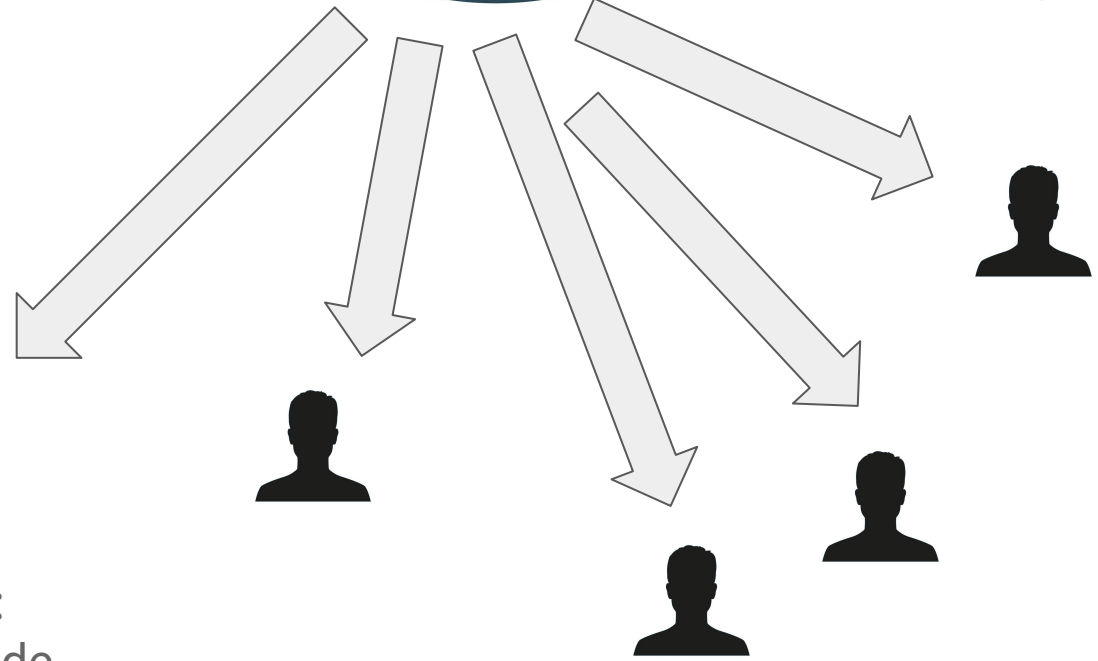
Spends years and several billion $$$ in engineering hours to make Google+, a huge flop



Creates the Facebook Like in a hackathon, one of the most influential features of all time

Staff Engineer:

- Comes up with impactful project
- Works with designers to create mocks
- Decomposes into swimlanes
- Puts together a fitting team
- Creates thorough system design



Junior Engineer:
Writes code

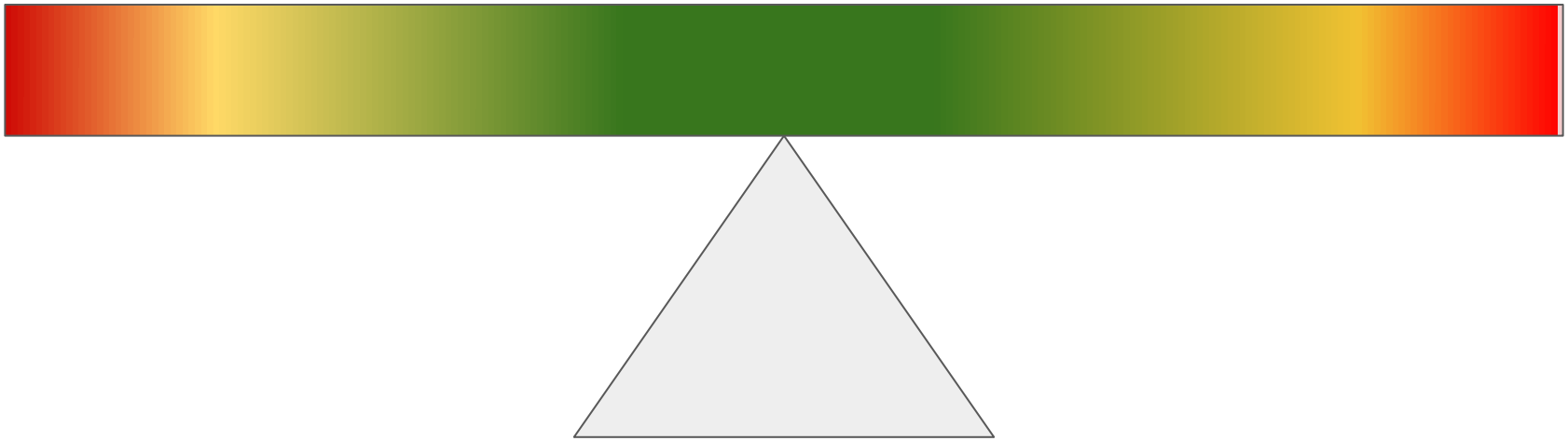
Promotion requires doing things that are fundamentally new to you.

Do It For The Right Reasons

Be obsessed with promotion and make it your sole motivation 😤

We want to be here ⚖️

Just do good work and naively hope for the best 😇



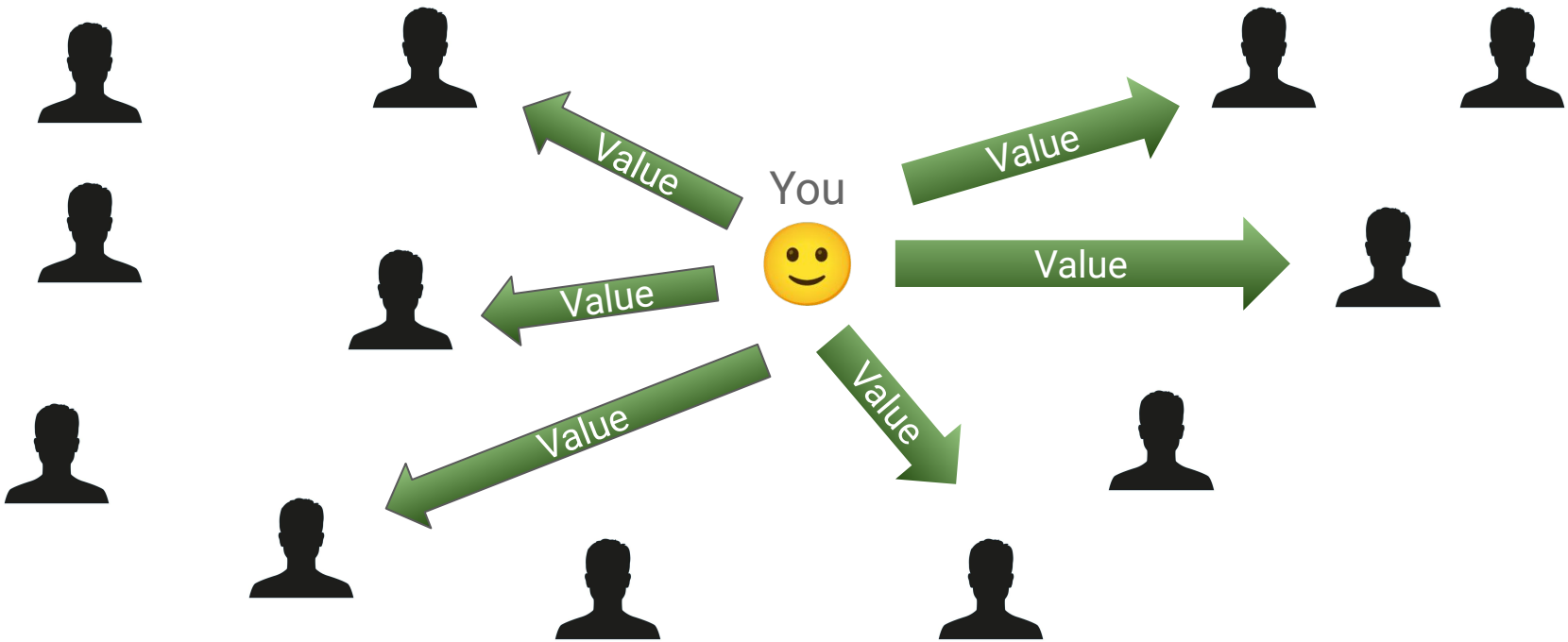
You should be **deliberate** about promotion..

...but you shouldn't let it **consume you**.

Growth comes first. 🪴

Growth comes first.

Higher Seniority = More Arrows ➡️



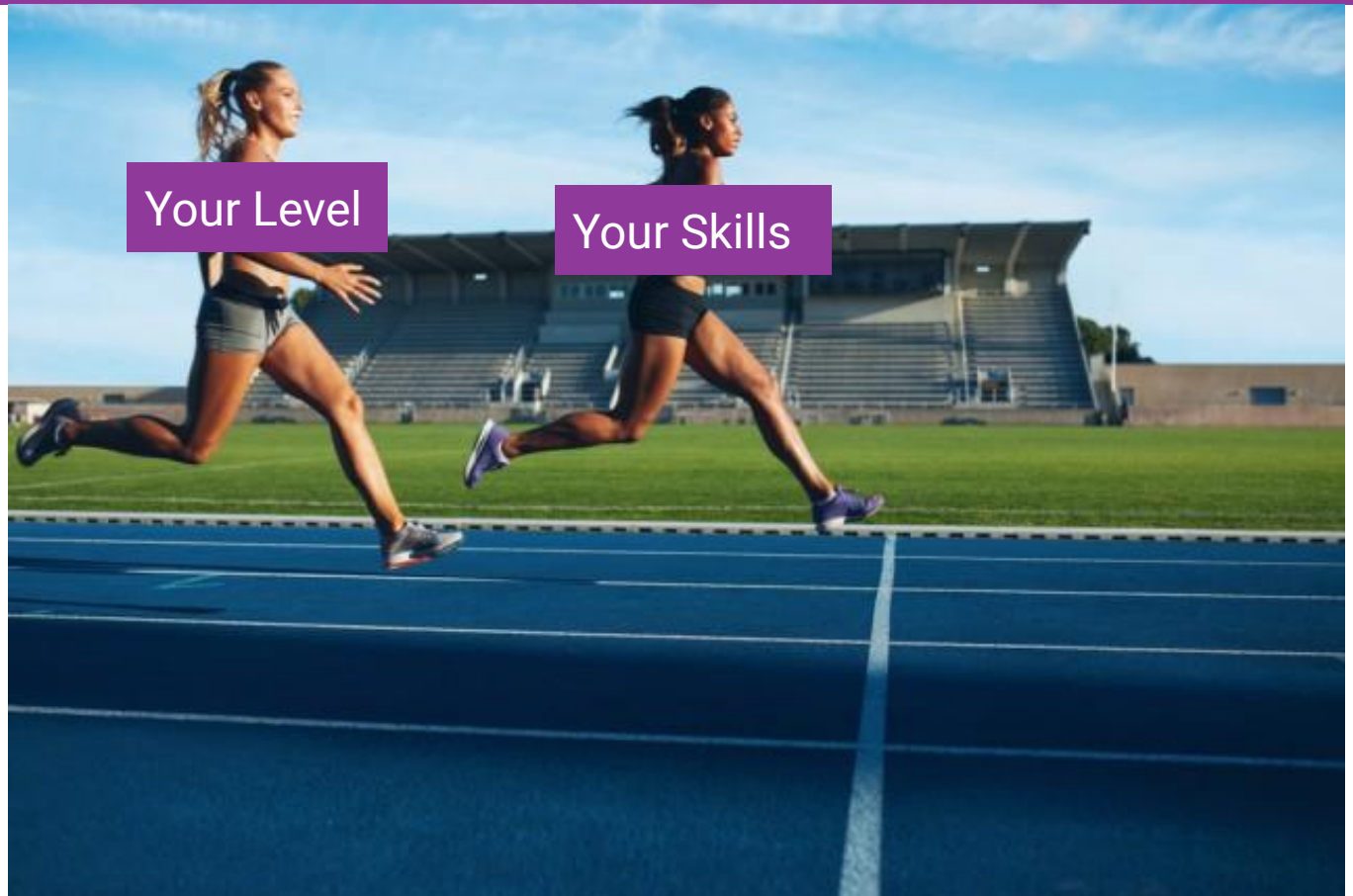
Promotion-Hungry

- *“This project is too low impact - I want my teammate’s higher impact one.”*
- *“You’re more junior than me, so it’s a complete waste of my time to help you.”*
- *“I need to take as much of this project as possible for myself.”*

Adding Value

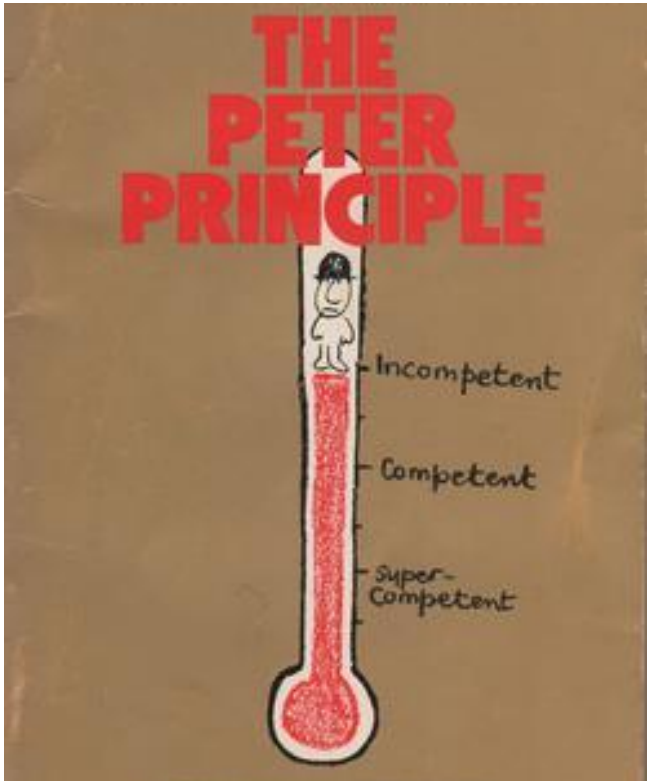
- *“How can I expand the scope of my project, so it’s the same as our team’s top projects?”*
- *“How can I give this junior engineer the tools to solve their own problems?”*
- *“There’s 2 of us - How can we achieve more?”*

Underleveled? It’s A Feature, Not A Bug



Promotions Are *Lagging*

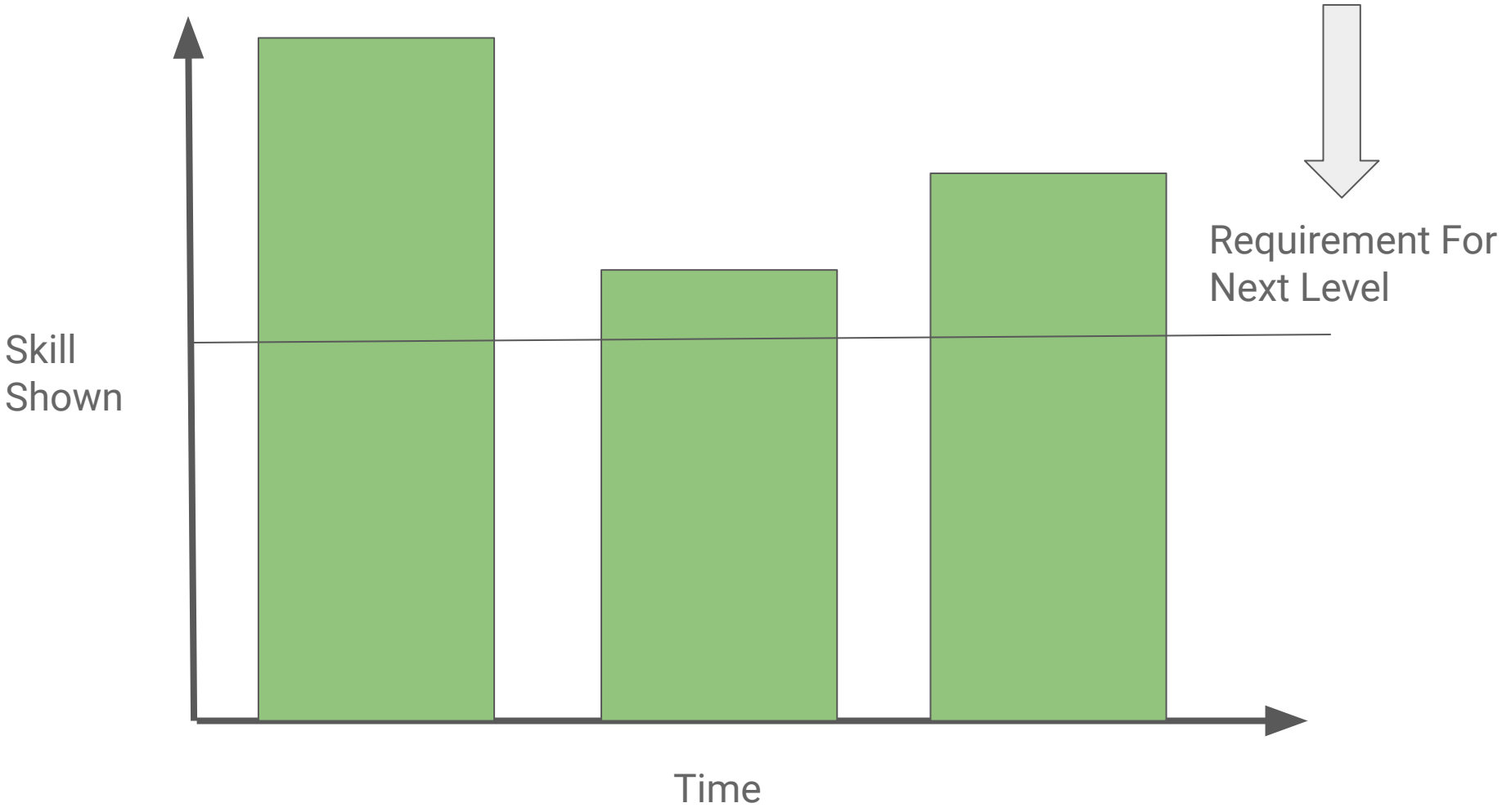
- Each promotion should involve a *meaningful behavior change*
- Mastering your current level != promotion
  - Doing this leads to “The Peter Principle”
- Promotion means you have already been functioning at the next level for a while



So Why Do We Have To Wait? 😫

1️⃣ Make sure that your skill floor is healthy

2️⃣ Verify the longevity of your work


Requirement For
Next Level
Skill
Shown
Time


SHINY PROJECT
SHIPPED ON TIME
CRIPPLING TECHNICAL
DEBT, NO LOGGING, HIDDEN BUGS
imgflip.com

What People *Think* Promotion Is

Level X

- Skill A
- Skill B
- Skill C

Level X + 1

- Skill A++
- Skill B++
- Skill C++

What Promotion *Actually* Is

Level X

- Skill A
- Skill B
- Skill C

Level X + 1

- Skill A++
- Skill B++
- Skill C++
- Skill D ✨
- Skill E 🌟

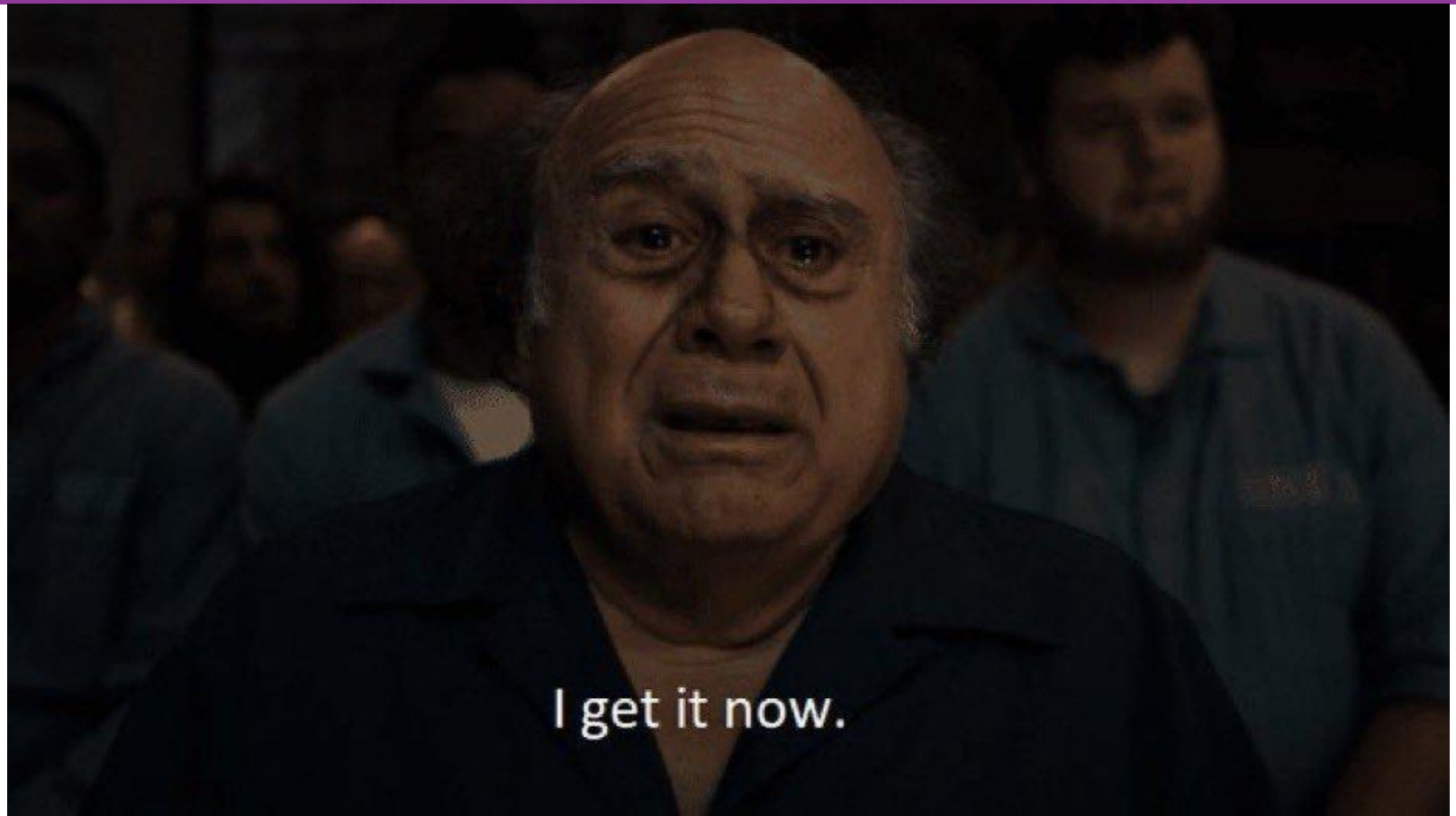
Mid-Level 🙂

- Write great code independently
- Fix tricky bugs in your tech stack
- Review teammates' code regularly

Senior 😎

- Set the example with stellar code that virtually never breaks
- Fix gnarly bugs spanning multiple tech stacks
- Review code across your team and sister teams, catching deep issues
- Lead projects end-to-end, owning relationships with XFN
- Come up with creative new projects

An Easy Way To Understand The Next Level

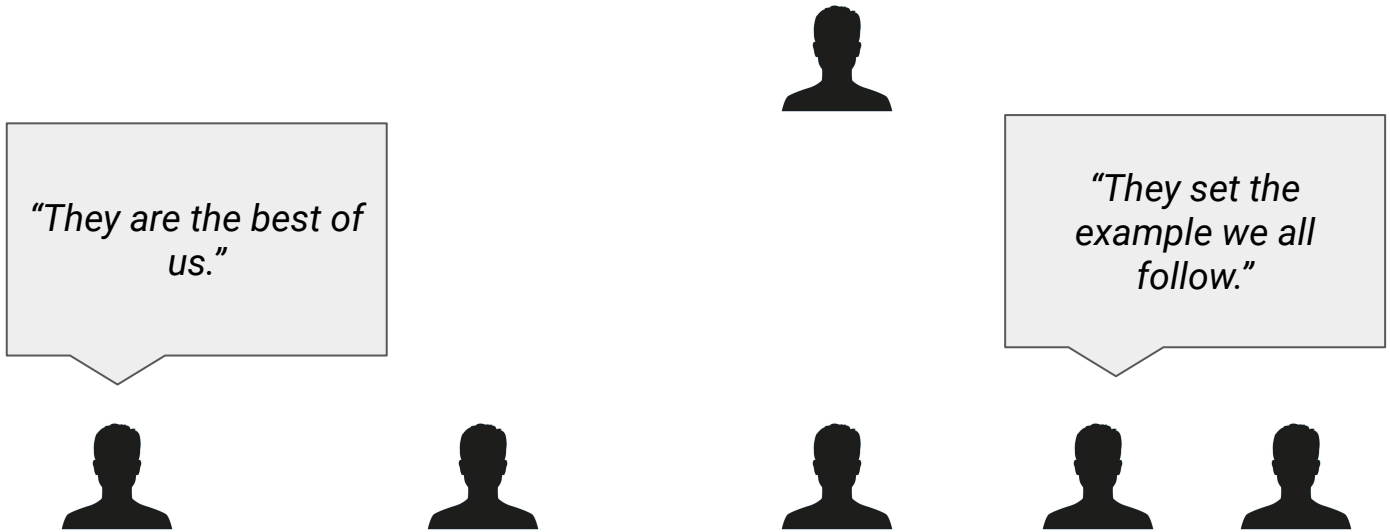


You are level **X**.

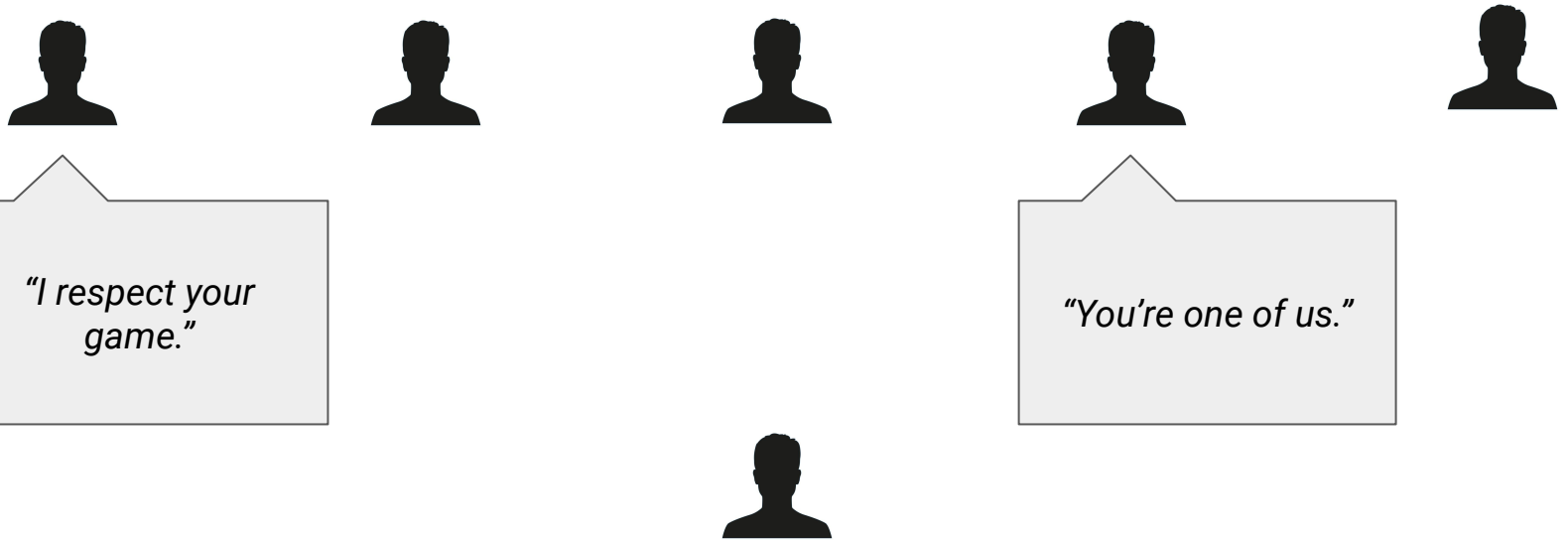
You want to get to **X + 1**.

So... how does that work? 🤔

Emerge As A Leader Among Level X



Considered As A Peer Among Level X + 1



What is everyone at your level focused on doing and how can you ascend past that? 

What are engineers at the next level doing that you aren’t? 

Finding The Right Team

The #1 Piece: Engineering Manager (EM)

Your EM Is The Most Important Person

- In 99% of companies, your EM is the most important player in promotion
- Roles:
  - Set expectations
  - Deliver feedback
  - Networking initiator
  - Boosts visibility
  - Vouch in perf review
- How to find a good one? 🤔

The best indicator of future behavior is past behavior.

“Wow, you went from junior to senior staff in just 5 years - How did you do that?”

“I uh... worked hard?”

Risky EM

❌ *Has never (or rarely) gotten an engineer promoted before*

❌ Relatively new to the EM role (<1 year of experience)

❌ 15+ reports

❌ External hire (i.e. they came from outside the company)

Strong EM

✅ *Stellar track record of getting engineers promoted fast*

✅ Veteran in the EM role (>3 years of experience)

✅ <=10 reports

✅ “Home-grown” talent, grew through company as an IC

Talk To Engineers

- EMs are inherently biased - Of course they want you to join their team!
- Get information straight from the horse's mouth
  - This is more feasible for internal transfers
- If you can't talk to engineers, try to find them on LinkedIn

Avoid Level Overload

The 2 Types Of Level Overload

1. **Same level** - Overly competitive

2. **Next level** - Not enough scope

- **Same level overload** is 1 of the most common causes behind sluggish promotion
- Most organizations have hard caps on how many promotions can happen
  - Often split by level
- Tends to lead to competitive teams where engineers fight each other
- Scope isn't 0-sum, but it's also not infinite

SDE II

L5

Junior	Mid-Level	Mid-Level	Mid-Level (You!)	Mid-Level	Mid-Level	Senior

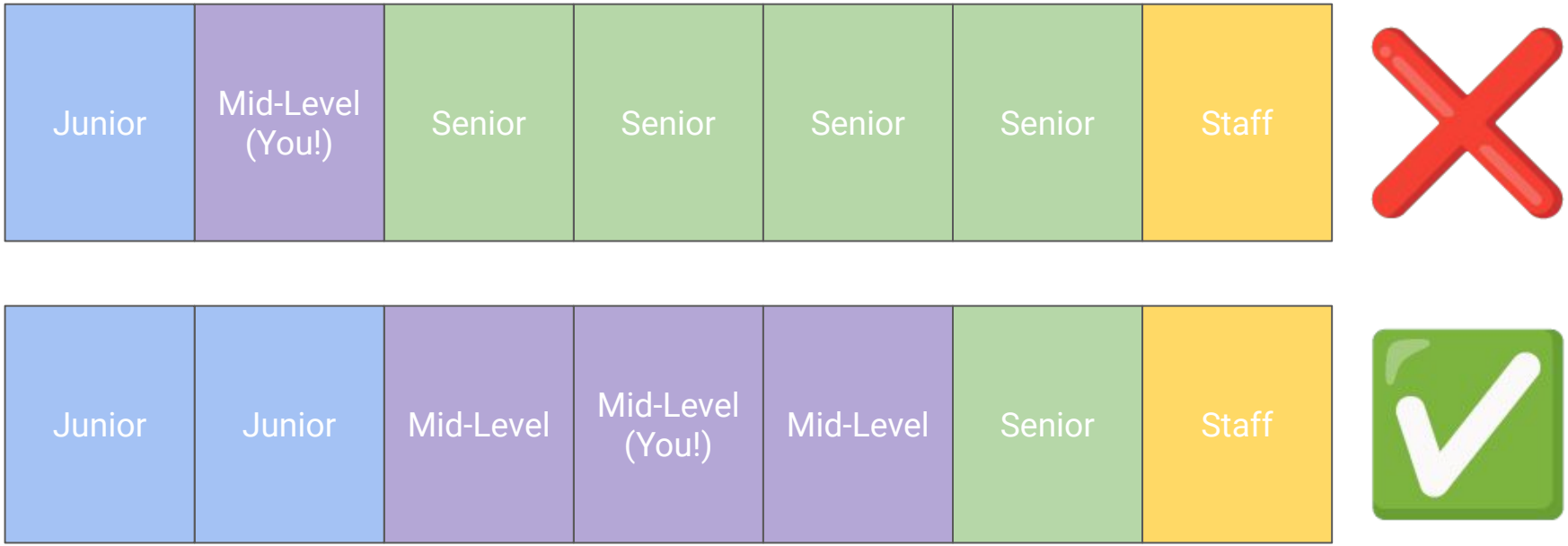
❌

Junior	Junior	Mid-Level	Mid-Level (You!)	Mid-Level	Senior	Senior

✅

Try to keep it below 50% ⚖️

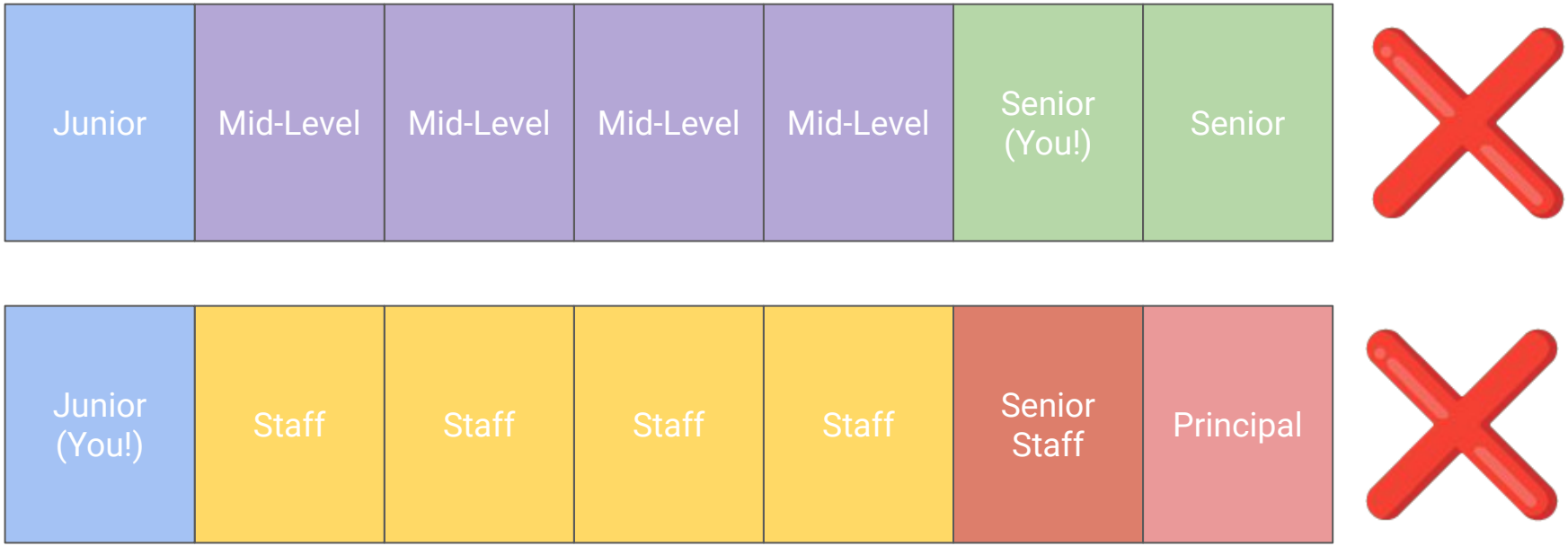
- **Next level overload** isn't nearly as common
  - Not nearly as bad as same-level overload
  - Potential silver lining is that there's a track record of uplifting people to this level
- The problem here is more around lack of scope
  - Really relevant when going for senior/staff+



Try to keep it below 50% ⚖️

You Need More Senior Teammates

- Most people understand this: You need role models
- Best case scenario is that one of these more senior engineers mentors you
- The ideal is X + 1
- Try not to go past X + 2, even +2 can be hard
    - They won’t remember what it’s like at X
- Higher-end of your level works too



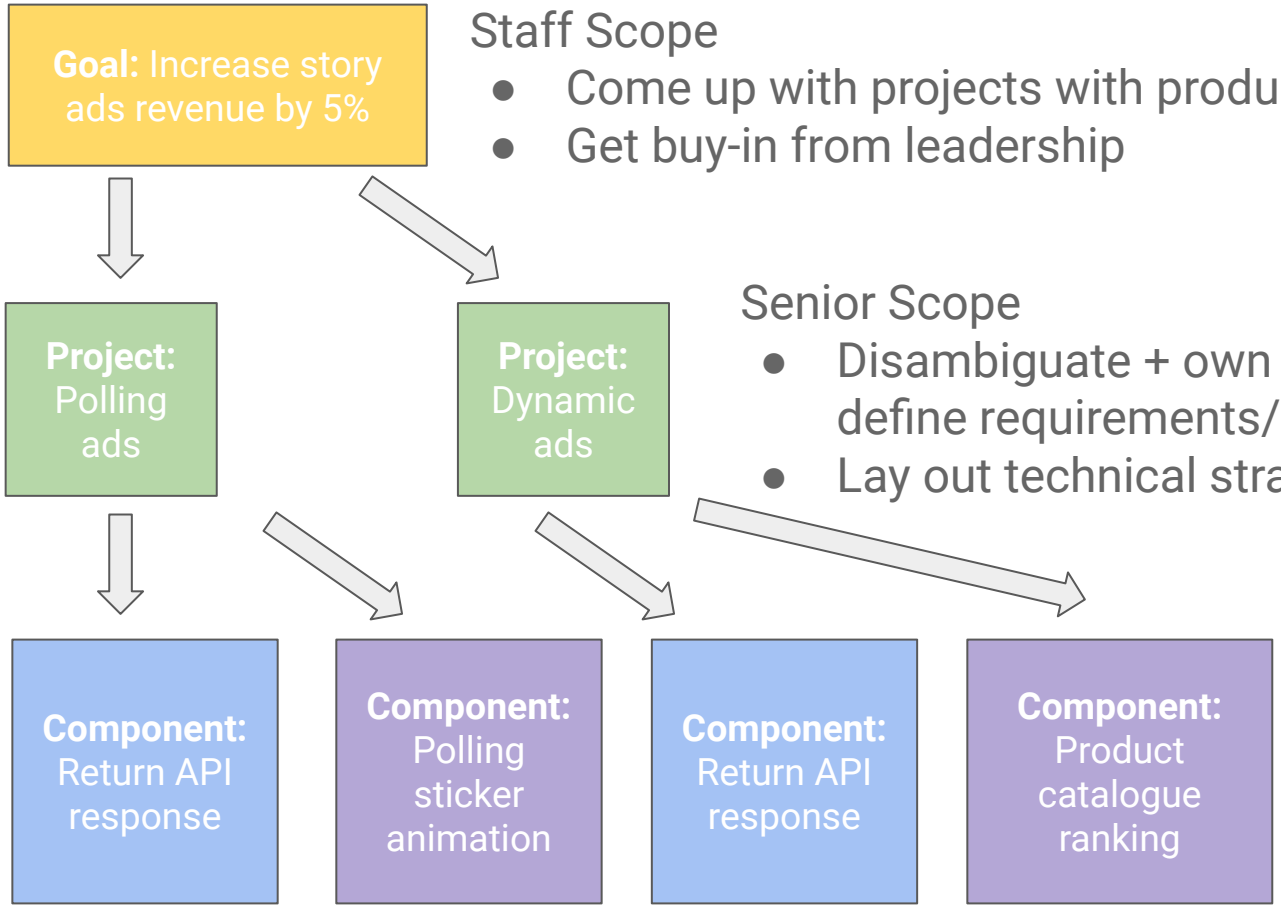
Don’t be too senior or too junior relative to everyone else ⚖️

Junior	Junior	Mid-Level	Mid-Level (You!)	Mid-Level	Senior	Staff

You want teammates who can empathize with you 🙏

You Need More Junior Teammates

- Many engineers don’t realize they need this
- For every team and project, there is a variety of scope
  - Some pieces will be more complex than others
- You need to be able to delegate out work at or below your level
  - Mentorship is really powerful here



Staff Scope

- Come up with projects with product sense + data analysis
- Get buy-in from leadership

Senior Scope

- Disambiguate + own project, define requirements/designs
- Lay out technical strategy

Junior/mid-level scope: Coding defined pieces

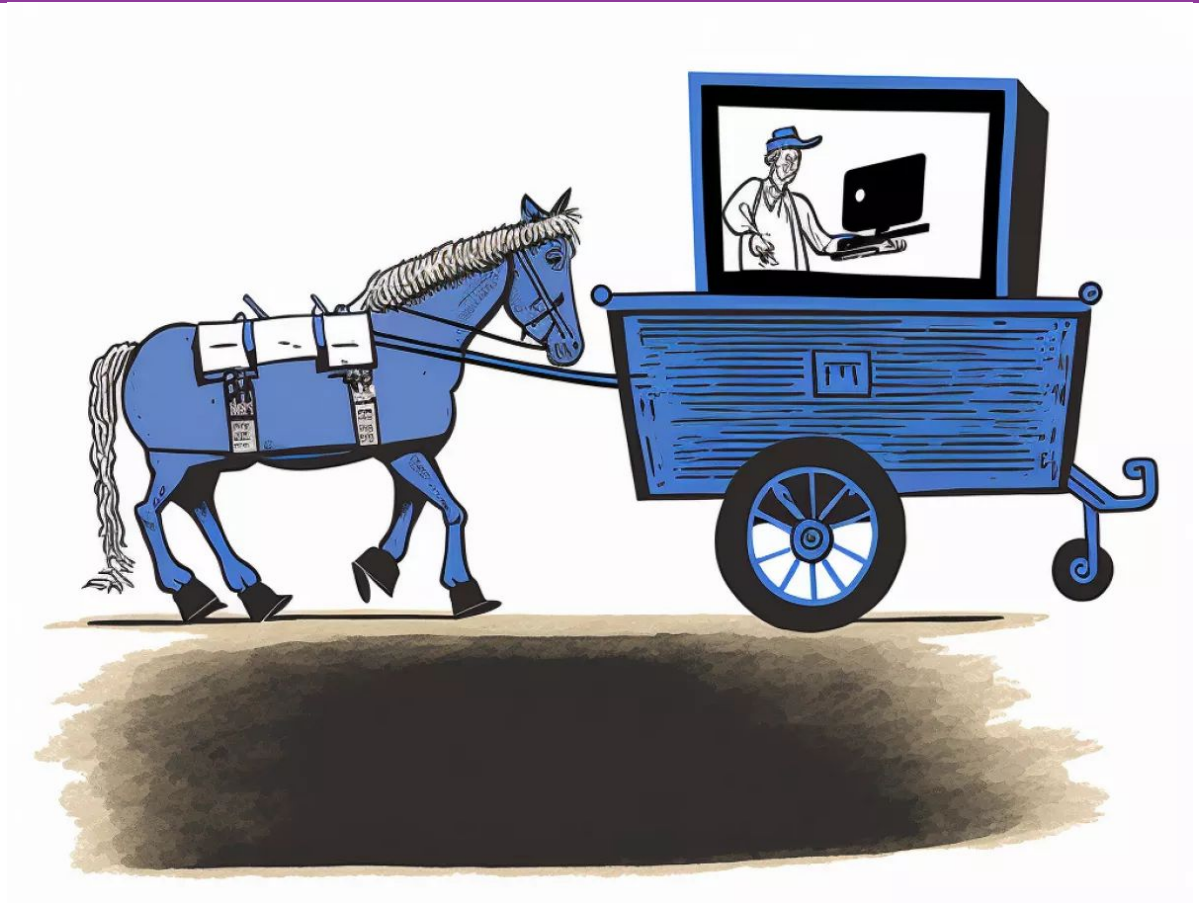
Mid-Level (You!)	Mid-Level	Mid-Level	Senior	Senior	Staff	Staff

Junior	Junior	Mid-Level	Mid-Level (You!)	Mid-Level	Senior	Staff

It’s generally great to be around the middle ⚖️

Set Up Success Criteria

Step 0: Establish Yourself



“I work hard, and I’m ready for the next level. Let’s talk promotion!”

“Yeah... You’re actually trending towards a PIP.”

- You should only bring up promotion if you have established yourself as a rock-solid performer
- It’s all about the feedback:
  - Manager
  - Peers
  - Performance review
- Wait *3-6 months* before starting serious conversations about promo

Talk To Your Manager

Step 1: Let Them Know

- *You have to be deliberate about promotion*
- It's not the manager's goal to get everyone promoted
  - It's impossible
- Frame the discussion as a *mutual benefit*: You want to expand your impact
- **Be empathetic:** Your teammates are asking too

Bad ❌

“I’ve been on this team for a while now, and I’ve been stuck at the same level. Can I get promoted?”

Good ✅

“I really enjoy working on this team. I want to do more to increase my impact and empower my teammates - What are the steps I need to take to get to that next level?”

“I would love to expand my role on the team - What steps can I take to get to the next level?”

“You’re doing great, but I think we need more time before seriously talking about promo.”

“Totally get it! Can we circle back to this 3 months from now?”

*If your manager doesn’t feel ready, get a **date for a date** 🗓️

Create An Expectations Plan

- Create a document outlining the expectations of hitting the next level
  - *Maintain it overtime with your manager*
- Make it as clear as possible - Illustrate the delta
- Split it up by performance review axes if your company has them (anchored to behavior)

You are the main person in charge of your own growth.

PLEASE CARRY ME

HERE'S MY PLAN TO BECOME EVEN BETTER - WDYT?

Take the initiative by starting the expectations plan with your best guesses and invite your manager’s feedback 📃

Bad Promotion Plan Expectation Example ❌

“Have a bigger influence in code review”

Good Promotion Plan Expectation Example ✅

*"**E4:** You are extremely thorough catching issues within your own domain, finding architectural issues (sometimes challenging the entire diff/diff stack), performance problems, and several edge case misses.*

***E5:** Instead of mainly reviewing stack-specific diffs within product domains you have worked on, you review diffs across the entire stack for your org (M2+ scope). You are able to review diffs for areas that you have never worked with before, quickly absorbing the context of the diff and its surrounding ecosystem to leave genuinely insightful feedback in a relatively short period of time."*

Meta E4 -> E5 Growth Plan

Expectations Breakdown

Keep in mind that **this is not a checklist**. The goal of this document is to provide levers for you to pull; every engineer has different strengths so they will get promoted in different ways. However, you will probably have to exhibit a good amount (50%+) of the behaviors outlined in this document in order to progress to the next level.

At high-level, going from E4 to E5 means transitioning from being a very competent IC leading small to medium sized tasks to becoming a strong leader, capable of pulling teams together and delivering end-to-end on entire projects, usually involving at least 7-10 people (XFN included). This is what it means to grow from a mid-level IC to a senior-level IC at a high-performance company like Meta.

Another attribute of E5 engineers is that they will start building an identity within their teams, teams being a mix of your product team and stack-specific team. E5 engineers will almost always have at least 1 thing (often 2-3) that they're uniquely good at, something that legitimately very, very few other engineers within their space can do. Part of your growth to E5 and beyond is figuring out what these special things are. What do you want to be known and respected for?

I want to call out that once you get to E5+, the picture gets more and more blurry as there's more paths to get to these levels and it takes much more to perform at this stage in your career. More specifically, the axes will start overlapping: For example, let's say you come up with your

Meta Engineering Performance Axes

1. Impact 🎯
2. Engineering Excellence 👨‍💻
3. Direction 🗺️
4. People 🤝

Engineering Excellence

As an E4, you demonstrate very strong technical proficiency within your space (i.e. your stack and team). As an E5, you need to build a technical presence that's both incredibly deep and extends outside of your immediate scope.

E4

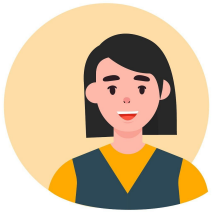
1. You build a reputation as someone who delivers a substantial amount of very high quality code. It is very difficult for folks to find problems in your implementation after it's launched.
2. You hold the quality bar for systems you have built. You make sure that the code you've written is always being extended in a responsible, scalable way instead of just "dropping and shipping". This largely manifests through going through relevant diff reviews touching your codebase like a hawk.
3. You continue delivering code in a timely, high-quality manner. For lower complexity projects, you build a reputation of delivering well ahead of time (25 - 50% faster than expected).
4. You care about test coverage. You may even lead small to medium-sized efforts (2 - 5 engineers) to add more test coverage to other parts of the codebase.
5. You have a strong presence in code review. You are extremely thorough catching issues within your own domain, finding architectural issues (sometimes challenging the entire diff/diff stack), performance problems, and several edge case misses. You are able to provide decent code review outside of your domain as well.
6. You have strong mastery over small (<1 month) and medium-sized (1-3 months) technical problems. You can deliver on larger technical efforts (3+ month time horizon), but you may need technical assistance scoping out these very large projects (3-6 month length or problems deep within the infra layer)

E5

1. You deliver code with extremely high-quality.
    a. You are often accounting for complex long-term issues that won't be a risk until 6-12 months out in your code. This shows from your reputation as someone whose code has very, very few SEVs.
    b. Your diffs are regularly referenced as a golden standard for what a great diff looks like, setting the diff culture for the team as a whole.
2. You are able to deliver code with extremely high velocity, even with medium to high complexity projects. You are often put on urgent, mission-critical projects, those with deadlines that are far more aggressive (30%+ less time) than the average project in your org.
3. You regularly solve large-sized technical problems (3 - 9 month projects). You almost never need hands-on technical assistance scoping out your work as you can thoroughly break it down and scope it out yourself.
4. You are working within medium to high technical complexity. This can mean:
    a. Working closer to the infra layer, solving problems like cold start and frame drops if you work on the mobile side for example
    b. Being further away from the product layer. When working on the product-side, you are often building common components with multiplicative impact (e.g. a button that's used everywhere) or on the hardest UI problems (e.g. manually drawing an animation frame-by-frame in Android, which is how the polling sticker in Instagram works).
    c. Working with massive, legacy codebases while still maintaining high-velocity and code quality.
5. You have a very broad and powerful presence in code review.
    a. Instead of mainly reviewing stack-specific diffs within product domains you have

Observe Other Engineers

- Identify high-performing engineers at your level and the next to learn from
  - Your manager can help with this
- Focus on the **how** and not the what: How is their *behavior* different than yours?
- In particular, observe how they react differently to you in similar situations



Staff Engineer

- Analyze user data, go through UXR, and do market research to identify opportunity
- Get buy-in from leadership with an expertly crafted presentation
- Work with product management and designers to define requirements
- Come up with system design and align the client-server protocol across sister teams
- Execute while clearing blockers and maintaining constant alignment

High impact project 💥

🗓️ What does their calendar look like?

💬 What kinds of people are they talking to?

🗣️ How do they contribute to meetings?

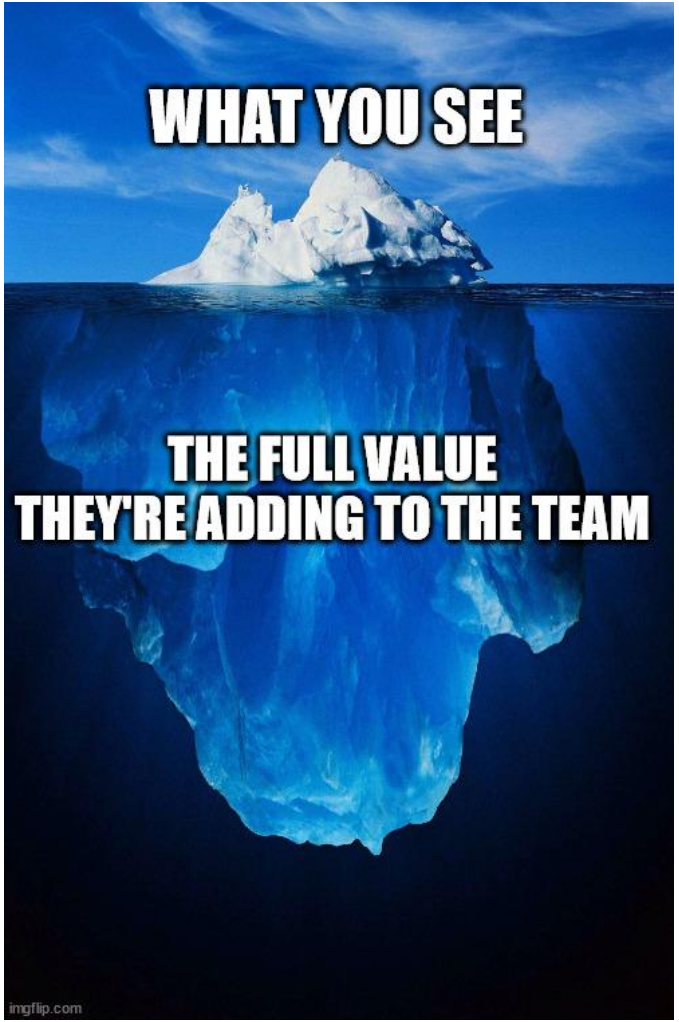
📃 How do they plan projects and design systems?

👨‍💻 Is there anything that makes their code and pull requests special?

Remember: Your goal is to **learn**, not compare yourself.

Avoid getting jealous 😟

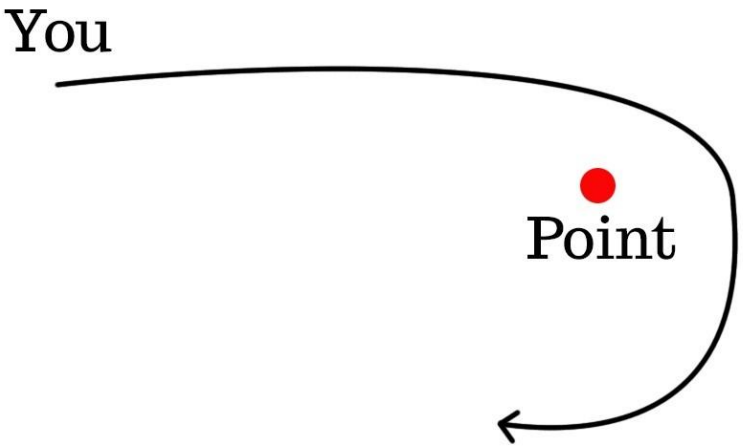
Very senior engineers tend to do a lot of “silent hero” type work. 🦸



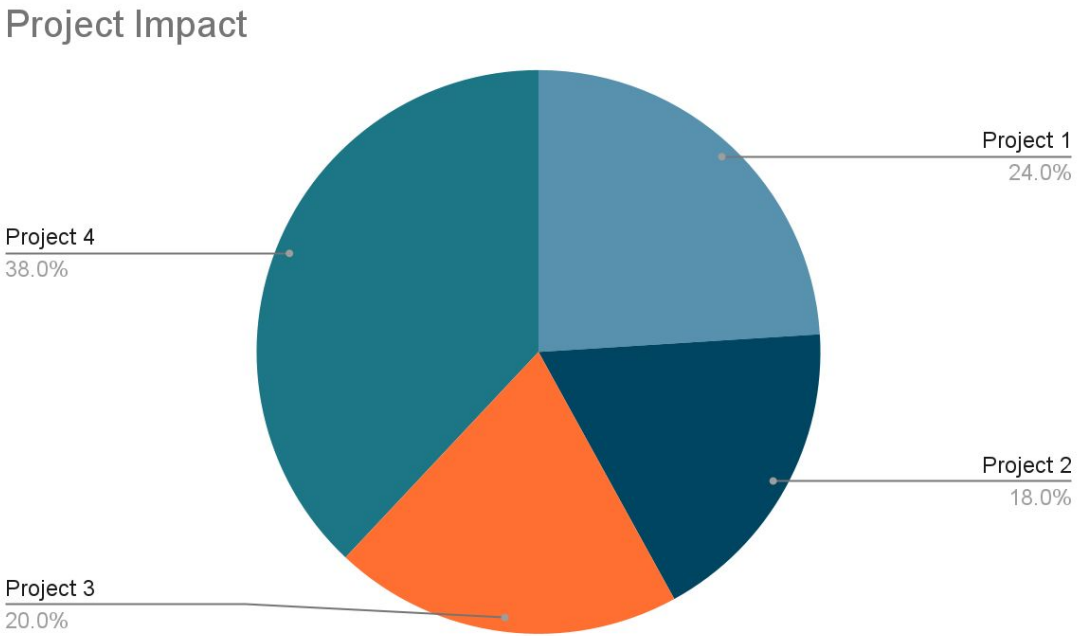
Project Credit And Scope

It’s Not Just About Impact

- *There is far more to how much you get rewarded for a project than just its impact*
- Impact has become a near meaningless buzzword
- Fixating on impact leads to a dangerous 0-sum mentality that ultimately hurts growth
- There are actually **6** angles when it comes to project credit (1 is impact)



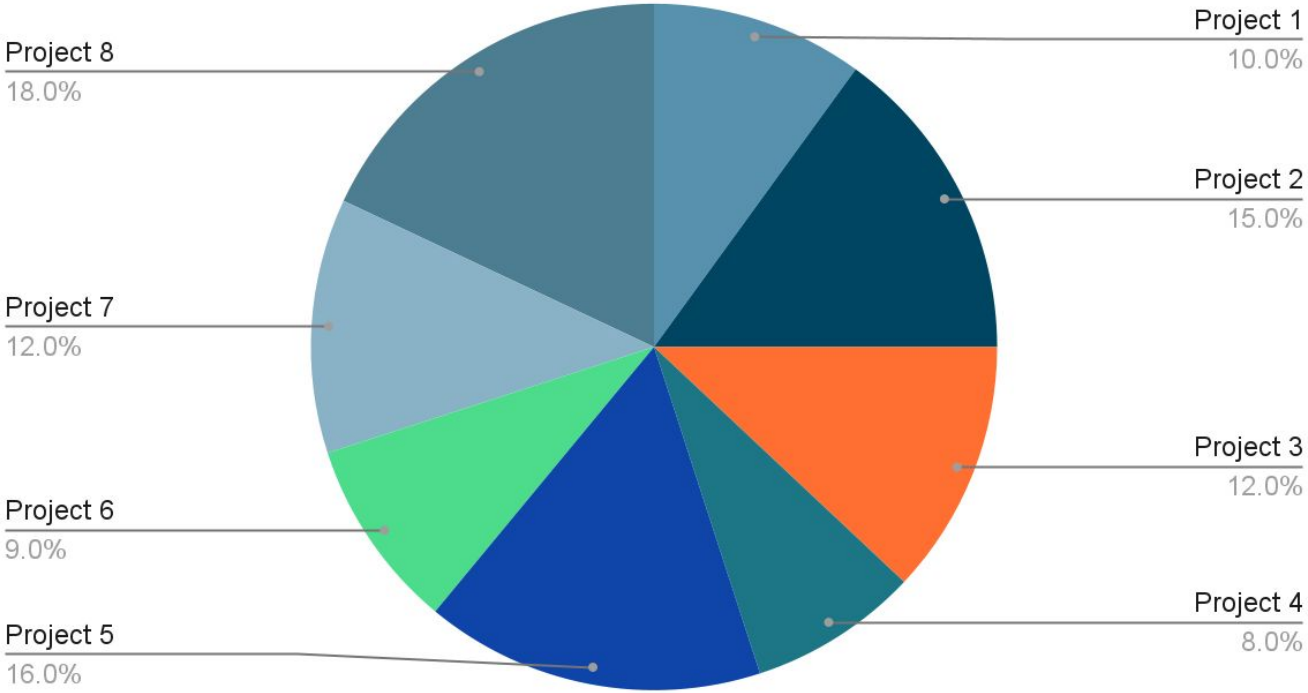
Unhealthy “Impact-Only” Mentality



“I will fight my teammates to the death to get Project 4.”

Healthy Holistic, Constructive Mentality

Project Impact



“How can we expand this roadmap to be more exciting?”

The 6 Project Axes

1. Impact 🎯
2. Technical Complexity 👨‍💻
3. Organizational Complexity 🤝
4. Depth Of Contribution 💪
5. Quality Of Delivery 👌
6. Attribution 💡

Impact

- The impact of a project is how much it contributes to your team's goals
- 2 broad types of goals:
  - **Metrics-driven** (common in bigger companies like FAANG)
  - **Build-oriented** (common for startups and younger orgs)
- If you don't have metrics, try adding them

OKRs = Objectives And Key Results 🔑

KPIs = Key Performance Indicators 📊

You should figure out what
these are for your team! 💡

Always ask:
Why?
🤔

Which Junior Engineer Gets More Credit?

- Has a **great** product manager
- Gets great projects that add massive user value and **hit goals**
- Delivers on-time and with high code quality

- Has a **mediocre** product manager
- Gets projects that usually miss and **don't contribute to goals**
- Delivers on-time and with high code quality

*They get the **same** amount of credit!*

Junior and mid-level engineers are generally not held responsible for impact. 🙏

Technical Complexity

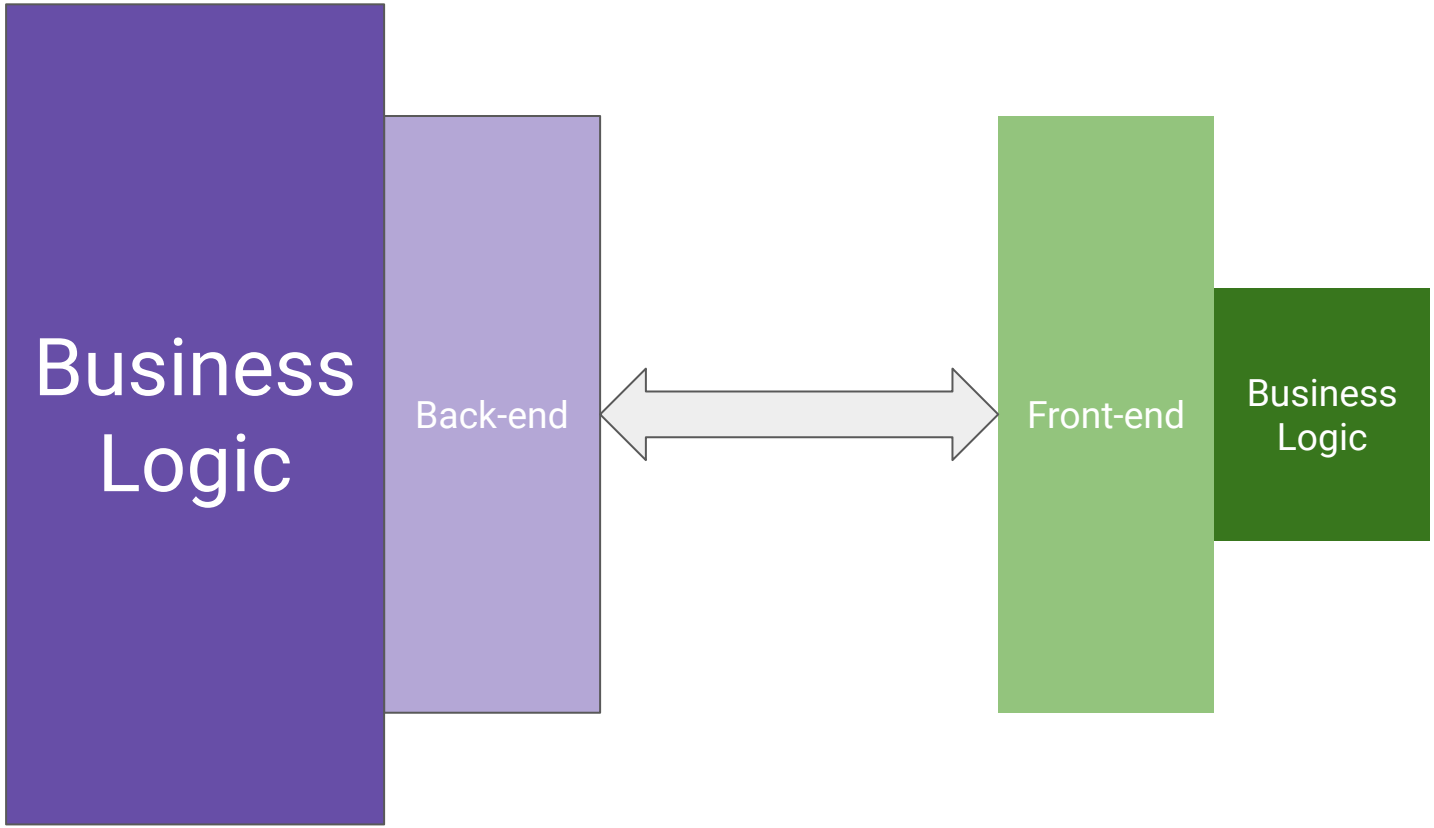
Not all code is created equal 🙅

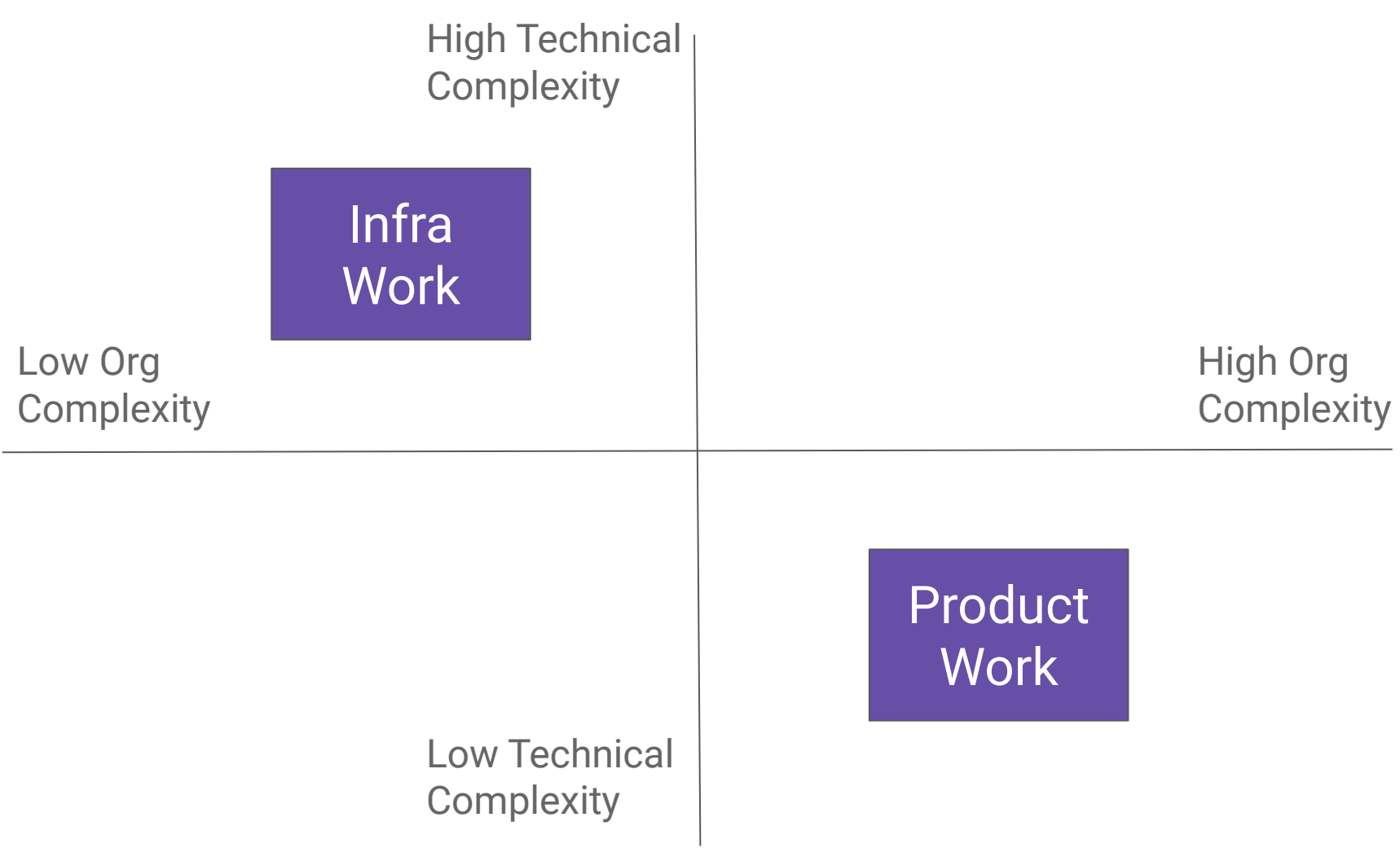
You can write **1,000** lines of code in **10** hours 🏃

You can write **10** lines of code in **1,000** hours 😩

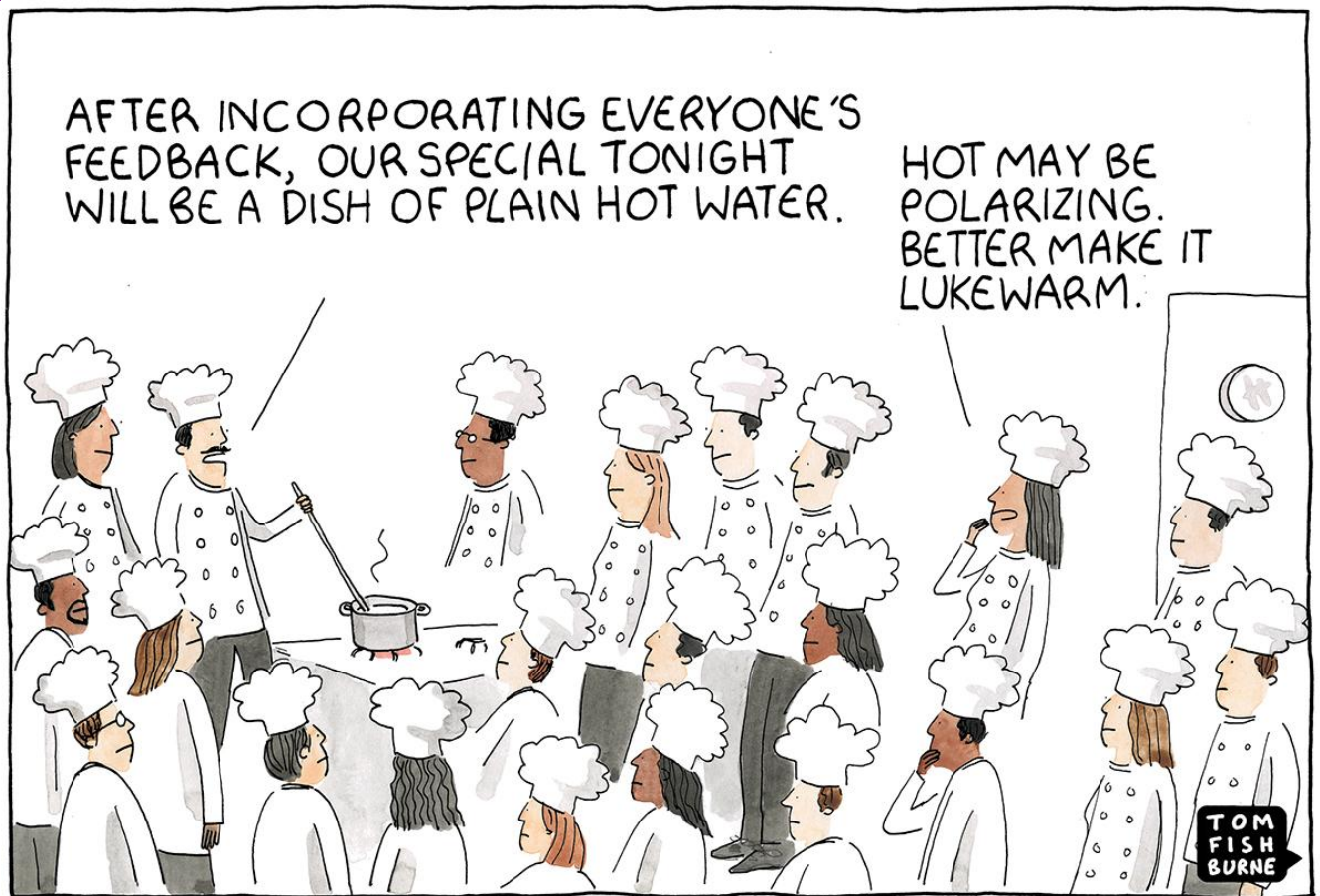
How Technically Complex Is It?

Dumb Client ✨, Smart Server 🧠

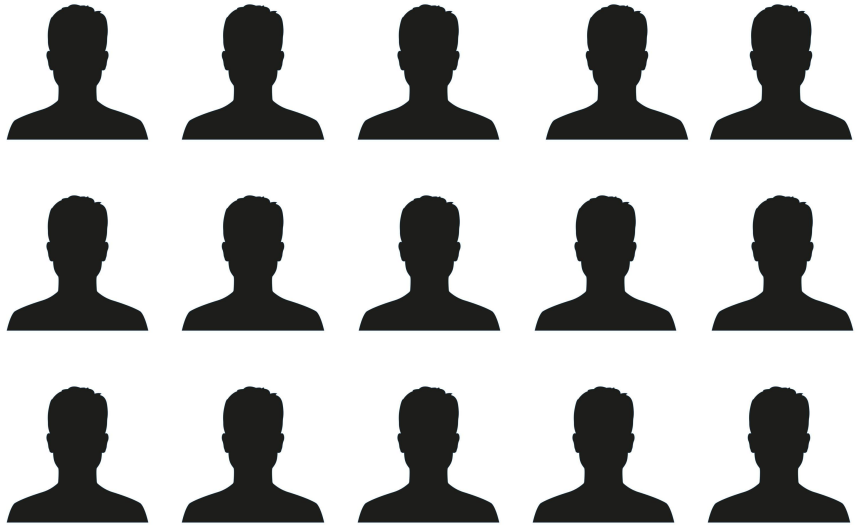



High Technical Complexity
Infra Work
Low Org Complexity
High Org Complexity
Product Work
Low Technical Complexity

Organizational Complexity



Who can throw your project under the bus? 🚌



Your Project

Low Organizational Complexity

High Organizational Complexity

Measuring Stakeholder Quantity

👩‍💻 How many engineers are writing the code?

✅ How many product managers worked together on defining the requirements?

🖼️ How many designers are behind the mocks?

🗣️ How many people are sitting in on the meetings?

Gauging Stakeholder Depth

👴 How senior are the stakeholders?

🤝 Are there engineers from other teams?

🤦‍♀️ How many non-technical stakeholders are there?

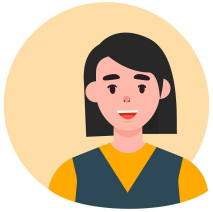
🔭 How distant are partner teams from yours in the company?

✨ Are you working with completely new partners?

Depth Of Contribution

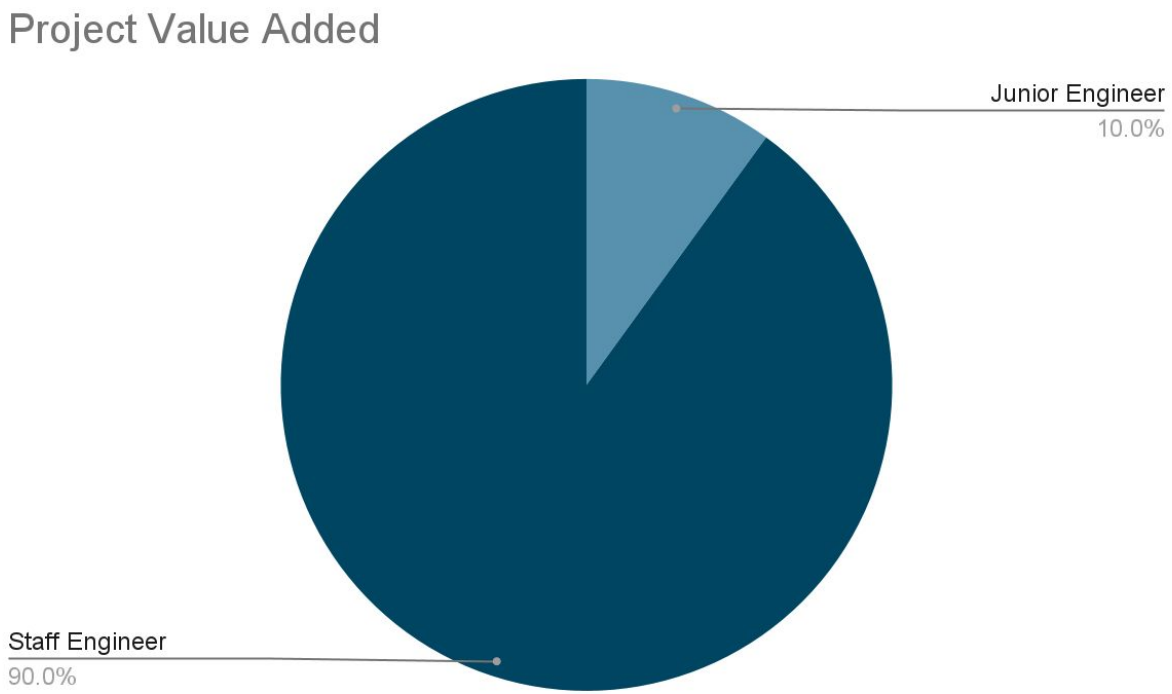
Junior Engineer

Wrote 2,500 lines of code



Staff Engineer

Wrote 100 lines of code



Why??? 🤔

How **hard** were the contributions you made to the project? 🤔

Would it be easy to find **another** engineer who could do what you did? 🫳

Were your contributions
something others are **averse** to? 🦸

Examples Of Hard Things

🔥 Heroically leading the team through a production issue

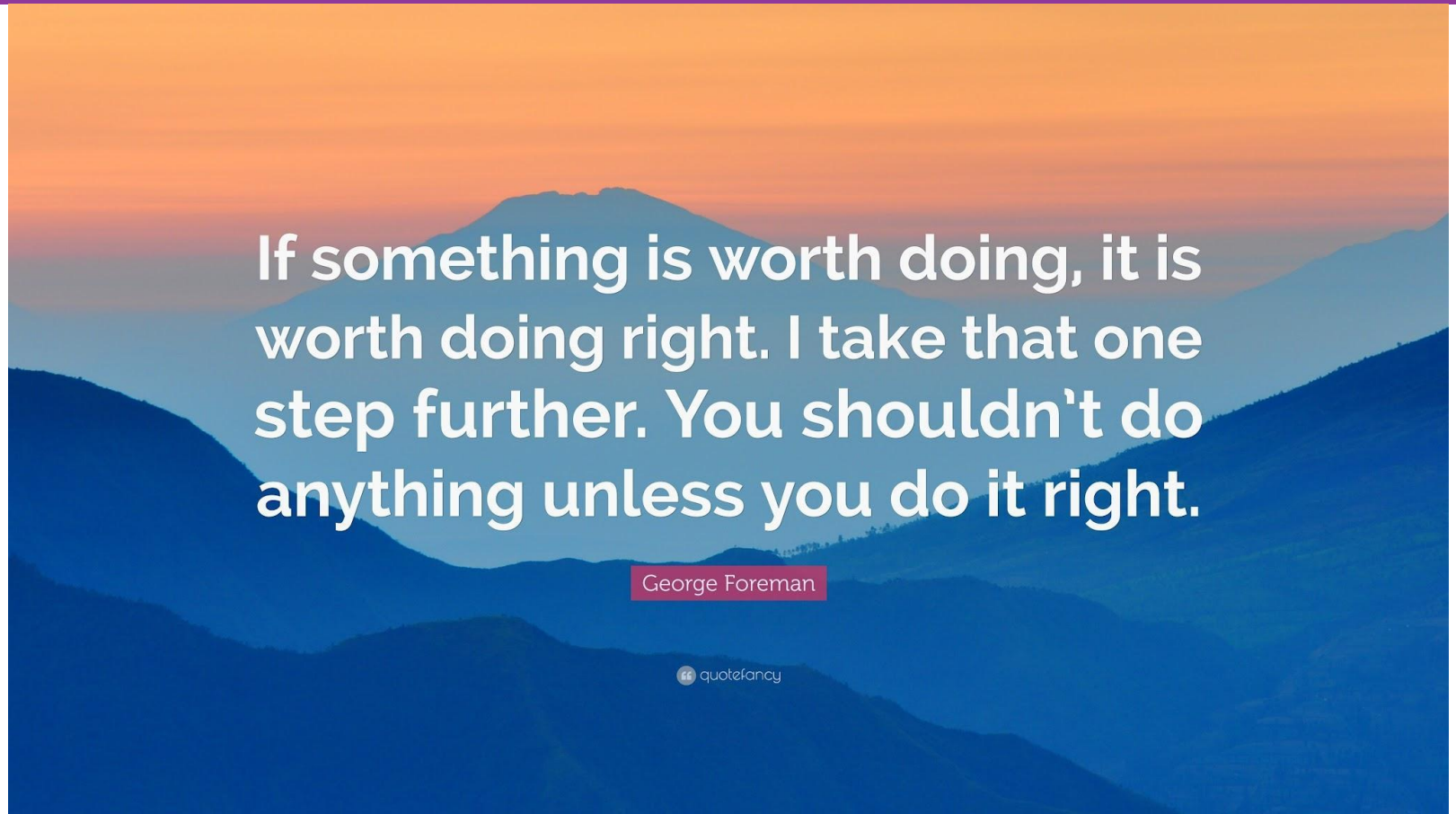
🗣️ Aligning a team that doesn’t see eye-to-eye with yours

✍️ Maintaining central documents in the face of thrash

🏃 Making the team faster without burning them out

🛡️ Shielding the team from XFN issues

Quality Of Delivery



- Getting impactful projects doesn’t matter if you completely fumble them
- Very senior engineers have reputations as “steady hands”
- There is tremendous value in being someone who never crumbles under pressure
- Leadership craves engineers they can count on

Quality Angles

⏰ Was it delivered on time?

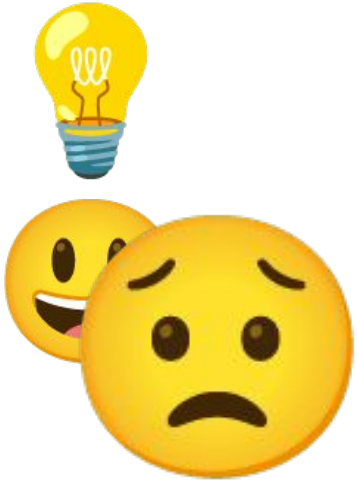
🐞 Are there a lot of bugs with the end product? Any production outages?

😵 How much thrash was there during the project?

🪴 Did you grow engineers along the way?
What about organizational relationships?

📊 Is the impact clear?
Is there logging and observability?

Attribution



I have a great idea!

Ideas on their own are worthless 🗑️

Idea ➡️ Project

1️⃣ Find the evidence to support your idea (UXR, data analysis)

2️⃣ Put together a compelling presentation

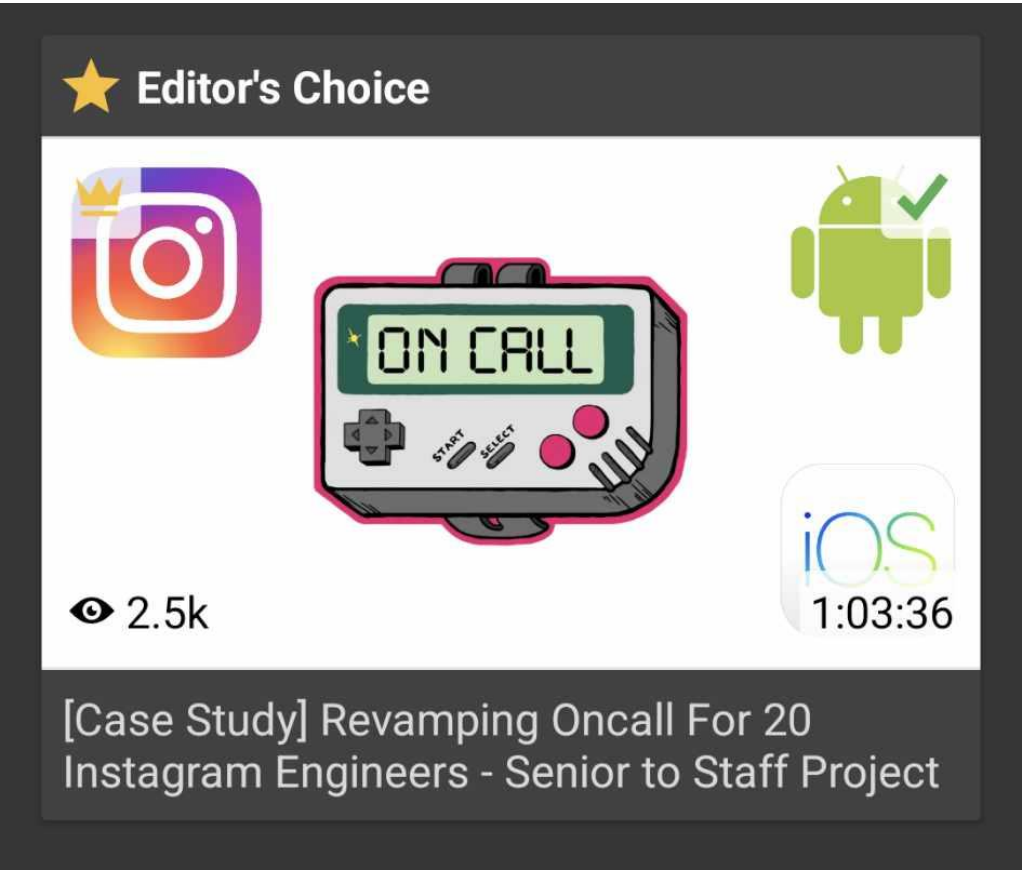
3️⃣ Get buy-in from leadership and other core people

4️⃣ Work with product to define requirements

5️⃣ Collaborate with design to bring it to life

6️⃣ Scope out engineering work (system design)

Example: Revamping Oncall For 20 Instagram Engineers

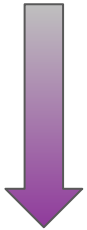


- Idea wasn't even mine, but I put in the work (99%)
- Leveraged years of cultivated social capital
- Aligned multiple EMs and senior/staff engineers
- Came up with a proposal and had to refine
  and defend it

The Project Credit Formula

Impact
Technical Complexity
Organizational Complexity

Depth of Contribution
Quality of Delivery
Attribution



Scope * Execution = **Credit** ⭐

There are *many* ways to get more points for promotion.

Be creative. 💡

Massive impact

Staff engineer technical complexity

15 engineers to lead across 3 teams

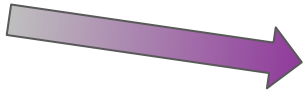
Write code and try to lead the project but needs to fall back on XFN to rescue

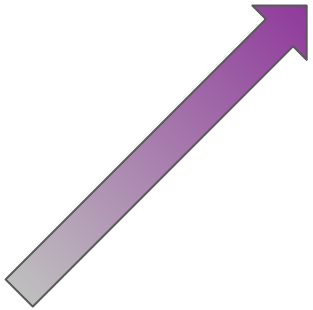
Ships 1 week late, tons of thrash, no observability, multiple production issues post-launch

Project handed to you by your engineering manager

Staff Scope

Poor Execution





Senior Engineer Credit And Behavior

Large impact

Senior engineer technical complexity

8 engineers to lead across 1 team

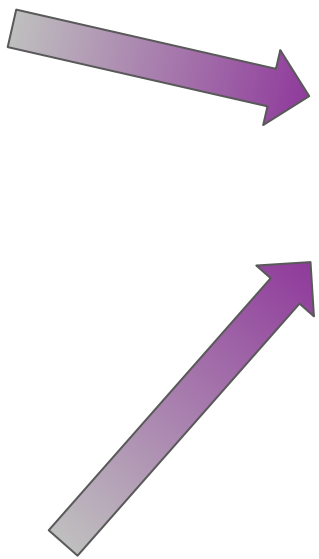
Write code, create system design, manage project, collaborate with XFN, do data analysis

Ships 30% earlier than expected, no thrash, great observability, 0 production issues post-launch

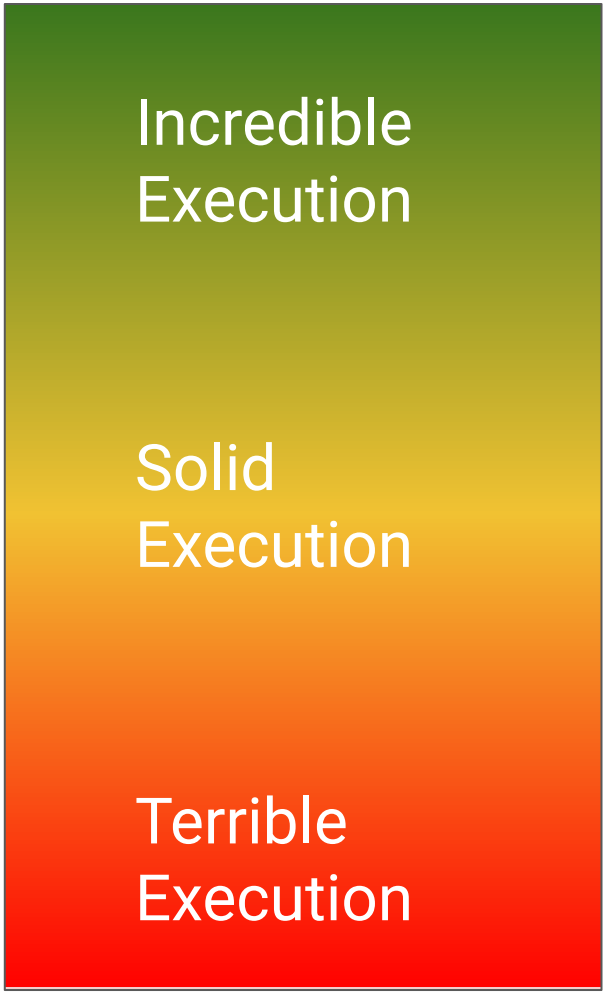
Came up with project on your own and expertly got buy-in from leadership

Senior Scope

Perfect Execution



Staff Engineer Credit And Behavior



Level X + 1 Credit 😎

Level X Credit 🙂

Level X - 1 Credit 😰

📝 Project Scope: Level X

Learn to get the most from the scope that you have 💡

Performance Review

Sell Yourself, Claim Your Wins

- You'll be surprised at how busy the most important people are (EMs, TLs, Staff+ ICs)
  - You need these people to give you good feedback
- *By default, assume that this group doesn't know what you're doing*
- Share an update with every accomplishment
- Don't constrain yourself to big launches - Milestones are nice!

Update Sharing Mediums

🗣️ Meetings (daily standup, *manager 1 on 1*)

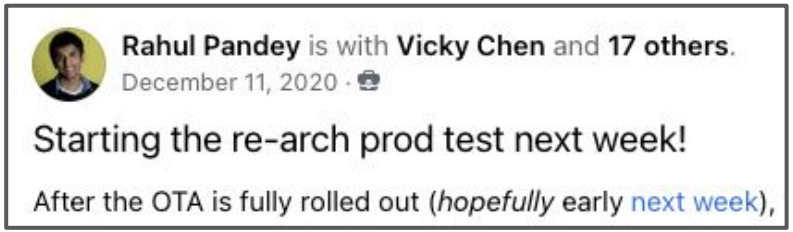
💬 Teams/Slack (smaller updates, modern companies)

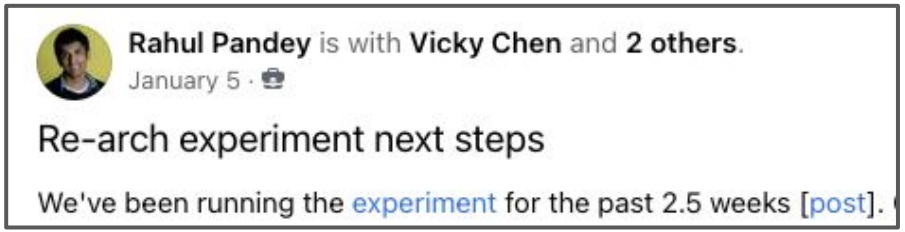
📧 Email (bigger updates, traditional companies)

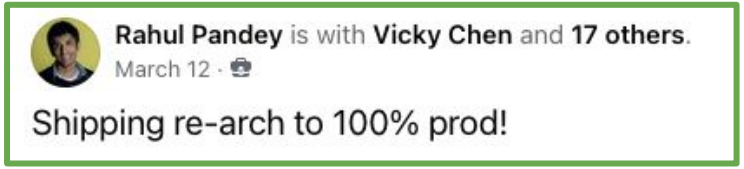
📊 Presentations (**huge** launches)

🏢 Workplace (if you work at Meta 😉)

Overcommunicate 📢







The **Many** Benefits Of Regular Project Updates

✅ Stakeholders are aligned (or realize that they aren't!)

✅ People are aware you're doing good work

✅ Provides onramp to get help with blockers

✅ Creates an artifact to link to
in performance review

✅ Boosts team morale by sharing thanks
and demos

Wednesday, March 13th

Alex C

I'm an idiot, so I pushed a new build yesterday that instacrashes when you open the video player. I upgraded the targetSdk from 33 -> 34, but I didn't test it on a device running 34 (my emulator was still on 33). I made the bad assumption the emulator would automatically upgrade to 34 after I upgraded Android Studio (like it does for iOS and Xcode), and I was obviously wrong 😂.

Anyways, **version 3.5.3** is going out now, and it should fix the issue. The fix was pretty much a 1 liner. Google Play is generally quick with minor app updates like these, so it should be out in production and ready to upgrade to in 2-3 hours. Sorry for the inconvenience everyone 🫡

(edited)

❤️ 12 2

Friday, March 15th

Alex Chiou 4:00 PM

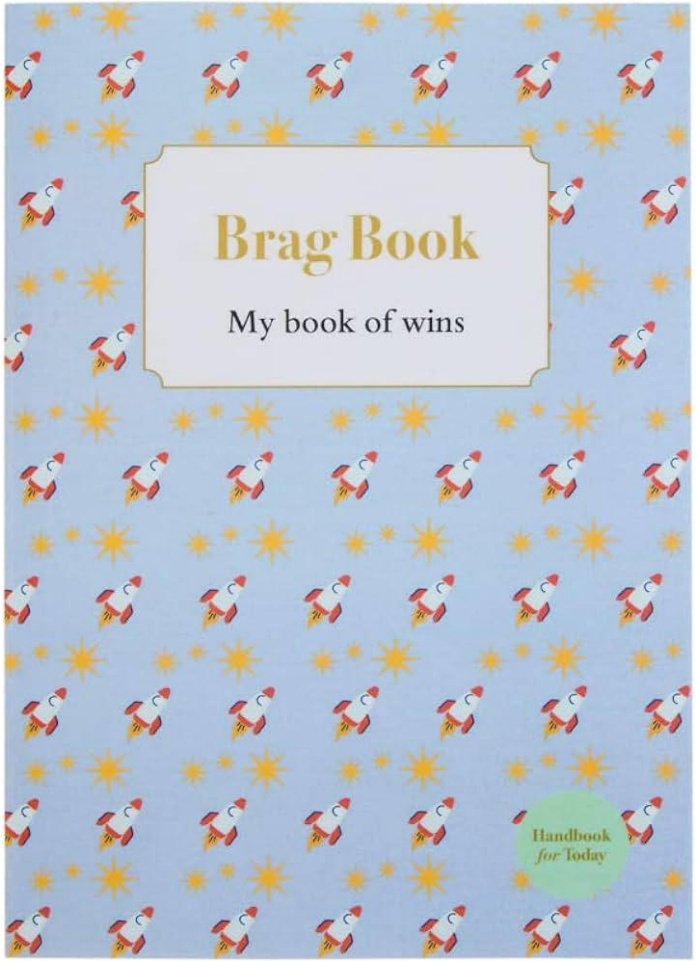
Support for reading 🧵 replies is now out across and iOS. I'll follow up on adding comment support so you can actually write replies in the mobile app as well. ✍️

Android Threaded Replies.mp4

You have to do the work to get full credit for your work.

Brag Journal

- Create a separate document detailing everything you're proud of
- Link the artifacts generated by selling yourself
- Many benefits:
  - Don't lose track of wins
  - Morale booster
  - Makes performance review much easier (self-review)
  - Quick "download" if you change managers



Lightweight Option: Manager 1 on 1 Meeting Notes

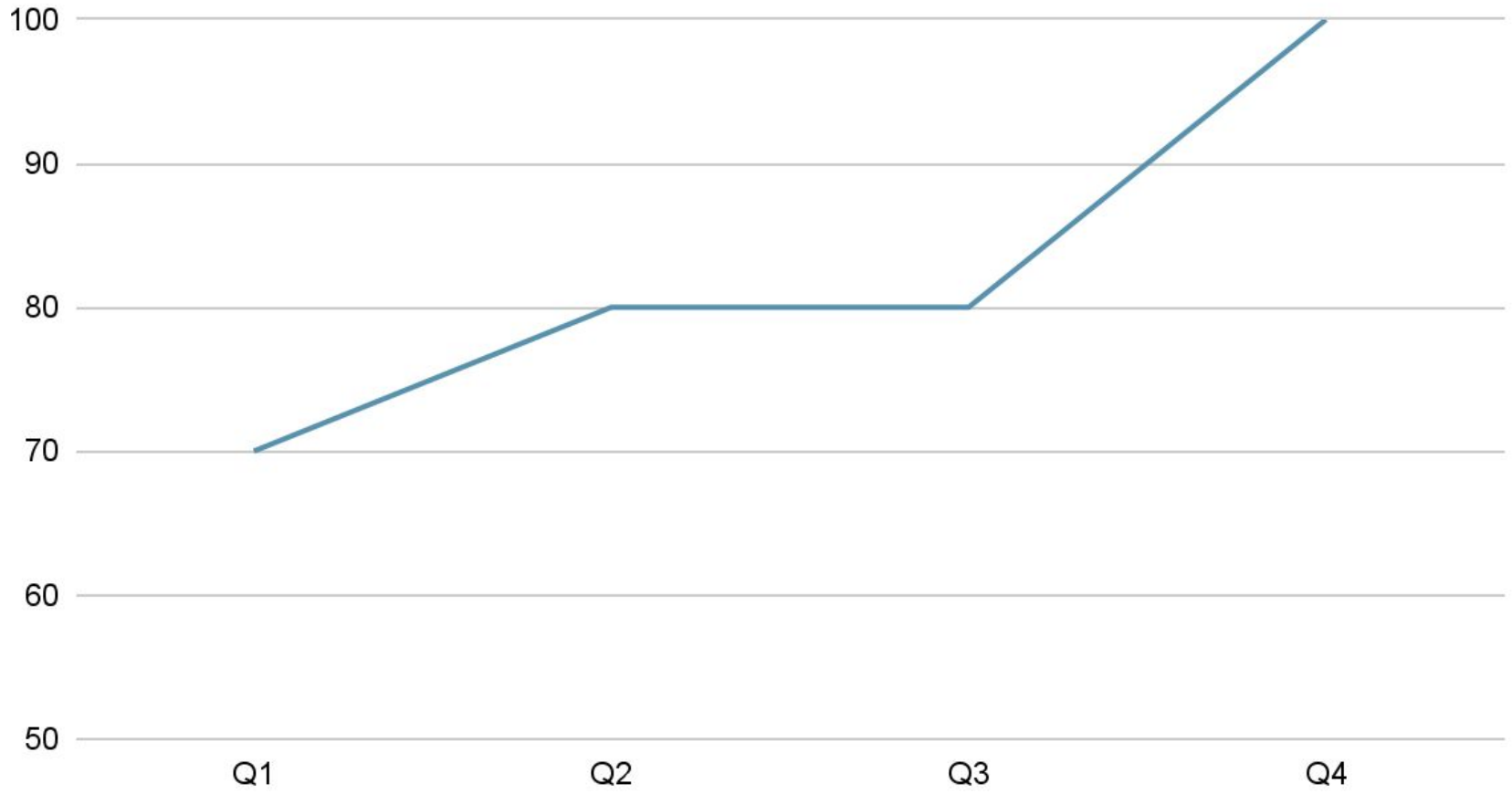
April 9, 2024

- Wins:
  - Shipped promotion course
  - Added support for threaded replies across Android and iOS
  - Onboarded the new mobile engineering intern (first PR!)
- Action items:

Timing: When To Amp Things Up?

- Humans have terrible memories, so they have substantial recency bias
- This problem is exacerbated the better your team and company are
  - Great engineering teams most fast and accomplish a lot, making it hard to remember earlier wins
- *Don’t go all out early - Optimize for a strong finish*

Your Performance Over Time



Peer Feedback Squad

“Will you write peer feedback for me in performance review?”

“Uhh, what did we work on together again?”

You

Staff Engineer

- Figure out who you want feedback from ASAP
  - Leverage your manager to find good people
- Work backwards from the signals you need and match peers accordingly
- After identifying the squad, collaborate with them, build the relationship, and do incredible work

TIME FOR A

THROWBACK

Peers at the **same level** should refer to you as a **leader** 🦸

Peers at the **next level** should refer to you as an **equal** 🤝

“X is a solid eng

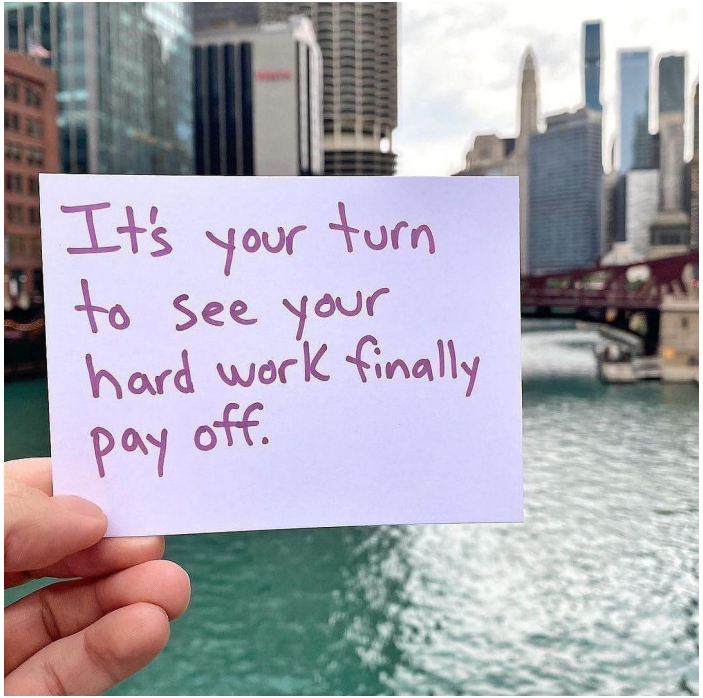
“X worked on p gh-impact win”

Same level: *“X sets the standard with what high-quality code review looks like with their thorough comments”*

Next level: *“X has excellent technical depth, debugging several issues even I had trouble with”*

Writing Your Packet

- This shouldn’t be too hard if you have been diligently following the prior advice
- Leverage the artifacts generated from your brag journal and overall sharing
- Anchor against your promotion expectations plan
    - Go through it point-by-point and make sure each is covered



“X could have a bigger presence in code review with deeper comments”

“X can code, but needs to do more technical planning upfront to catch issue”

“X should share their expertise by growing others”

“I reviewed 2x more diffs compared to last cycle and leave thorough feedback [DIFF EXAMPLES]”

“I shipped project A and B with 0 production issues by doing in-depth system design [LINK TO DOCS]”

“I mentored teammate C and got they got Exceeds Expectations for the first time in mid-cycle review”

Conclusion

This Might Not Be Enough

It’s possible to do **everything** right for promotion...

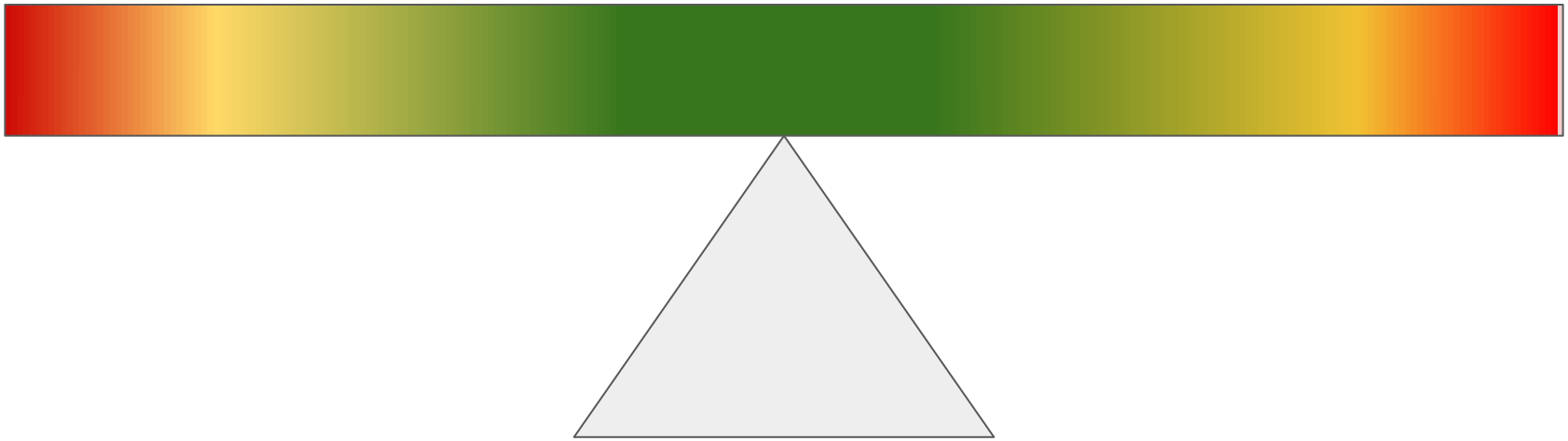
... and still not get it 😭

Now we have a dilemma 🧐

Stay on a bad team for too long and stagnate 🫠

We want to be here ⚖️

Switch teams constantly and never make progress 😞



The Large Cost Of Switching Teams

❌ Lose relationships with teammates, especially manager

❌ Lose muscle memory from tech stack mastery

❌ Lose deep understanding of culture

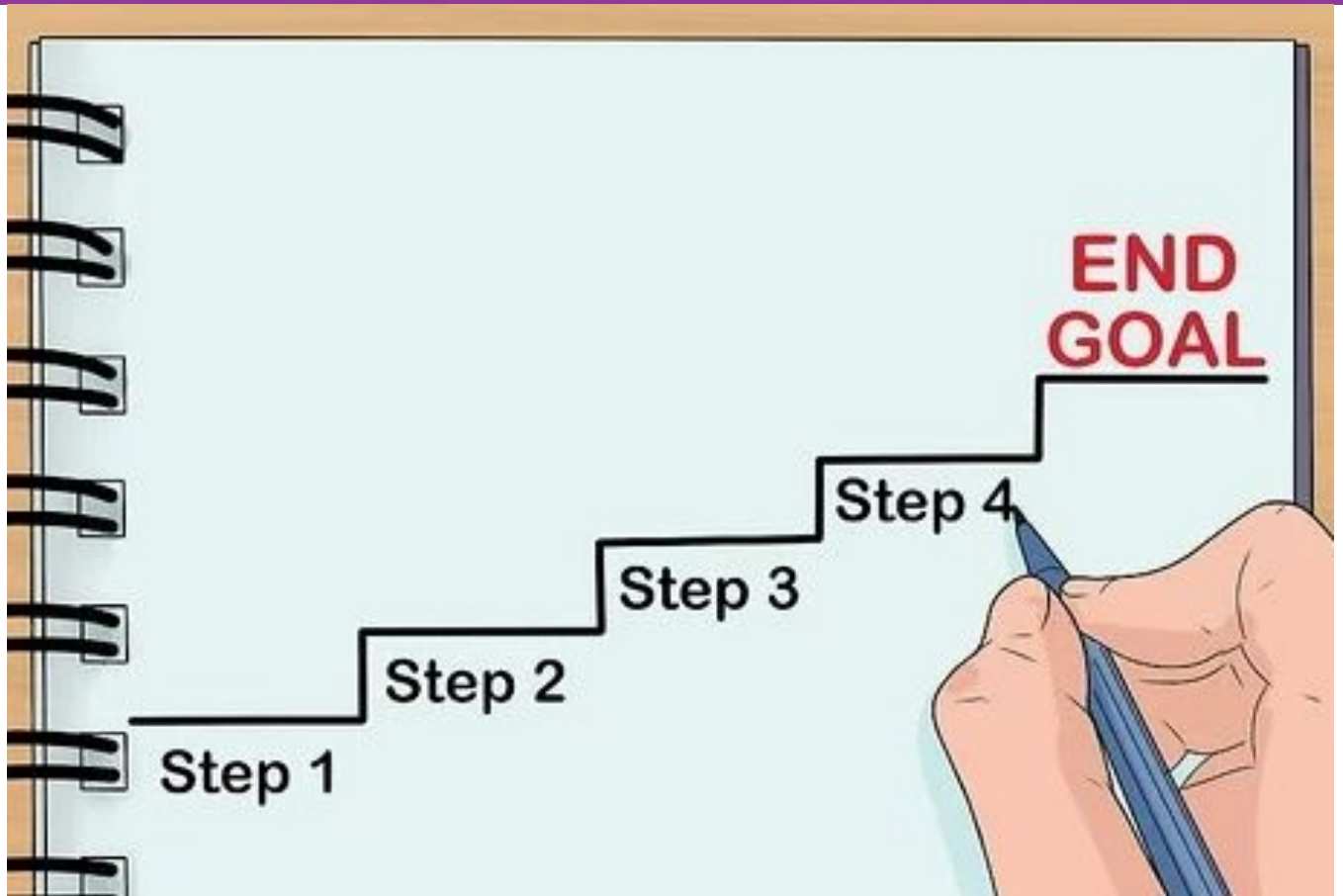
❌ Stop developing real engineering skills during interviews

❌ Spend 3-6 months onboarding onto new team

Should I Switch?

1. How many times has the contract been broken?
2. Is your org thrashy with manager/TL changes?
3. Is your company in an overall downward spiral?
4. Are you getting promoted slower than your peers across the company?
5. How’s the job market? Are you getting inbound?

The Step-By-Step Plan



1. Find a good team
2. Establish yourself as a high performer
3. Clarify expectations and gaps
4. Execute and grow
5. Go through performance review

Find A Good Team

- ❑ **Engineering manager has a track record getting engineers promoted**
- ❑ Teammates 1 level above you to learn from
- ❑ Teammates more junior than you to lead
- ❑ Team isn’t overloaded with engineers at your level
- ❑ Team isn’t overloaded with engineers at next level

Establish Yourself As A High Performer

- ❑ **Build a strong, honest relationship with your engineering manager**
- ❑ Identify core peers and add value to them to make them like you
- ❑ Develop a consistent track record shipping with velocity and quality
- ❑ Look for opportunities to expand scope of assigned projects
- ❑ Be creative and brainstorm new project ideas

Clarify Expectations And Gaps

- ❑ **Write a promotion plan with as much clarity as possible**
- ❑ Have that honest, deliberate conversation with your manager
- ❑ Observe engineers at the next level to understand the delta
- ❑ Break down your weaknesses across engineering axes
- ❑ Don’t get jealous of more senior peers

Execute And Grow

- ❑ **Make sure to check in on and maintain your expectations plan**
- ❑ Publicly document and share your wins
- ❑ Go through your gaps 1 by 1 and patch them up
- ❑ Be hungry for feedback and learn from mistakes
- ❑ Amp it up in the final quarter before review

Go Through Performance Review

- ❏ **Thoroughly go through your promotion plan and make sure it's all addressed**
- ❏ Compile artifacts from your brag journal, launch posts, and other sources
- ❏ Link as much evidence as you possibly can
- ❏ Request targeted feedback from core peers
- ❏ Share self-review with manager beforehand

Be Patient

Getting Promoted Is *Hard*

🪴 Repeated behavior change over a long period of time

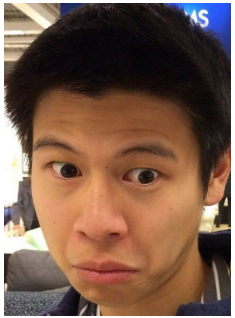
💸 Promotion budgets are cut in weaker economies

📃 Maintain growth plan as things change

😌 Be humble and accept your flaws

🕐 Promotions are naturally lagging

Focus on improvement.



n00b Alex

- Panics
- Waiting for instructions
- Reactive instead of proactive
- Missing the bigger picture
- Just coding up what's known

Insanely Ambiguous & Technically Complex Project

Tech Lead Alex

- Cool and confident
- Takes ownership as decision maker
- Proactive instead of reactive
- Views problem holistically
- Charges into the unknown

Do you see yourself making smarter decisions? 💡

Promotion Is A Journey: Follow Through

So many skills to grow...

Code review

System design

Mentorship

Project management

Creating scope

Data analysis

1 on 1 meetings

Scaling yourself

Be more active in meetings

A/B testing

Code velocity

Lead more meetings

Debugging

Writing concise project updates

Don’t get overwhelmed. Pick 1, improve it, move on. Repeat. 💪

Promotion is about behavior, not output.

Do the work to get credit for your work.

In order to grow, you must plant roots.

There’s more to it than impact.

Be deliberate. Have a plan.

0

Ask The Instructor: [Course] Nail Your Promotion As A Software Engineer

Alex Chiou (Tech Lead @ Robinhood, Meta, Course Hero)
2 minutes ago

Alex Chiou | Editor's Choice | Promotion

Promotion is one of the most difficult and nuanced topics, so there's no way any course can absolutely cover 100% of all possible scenarios. If you have any questions about the course, please ask them here! I'm happy to talk about:

- What gaps you may have to get to the next level
- How to create additional scope in your team
- Whether a prospective team you're considering looks good for promotion
- Why you're getting stuck at your current level
- Maintaining that dialog with your manager about promotion

What next?

- Share + get support: joinTaro.com/questions/create/
  - 💭 Ask questions in my instructor AMA for this course!
  - 🧠 Get 10x better advice than anything on Blind or Reddit
- Find community: joinTaro.com/events/
  - 👩‍💻 Join me for Group Office Hours!

team+nailyourpromo@jointaro.com